SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Computational Science and Engineering

Fast Multipole Boundary Element Method for Finite Periodic Structures

Abhijeet Abhay Sutar

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Computational Science and Engineering

Fast Multipole Boundary Element Method for Finite Periodic Structures

Schnelle Multipol-Randelementmethode für endliche periodische Strukturen

Author:	Abhijeet Abhay Sutar
Examiner:	Steffen Marburg, Prof. Dr.
Supervisor:	Arne Hildenbrand, M.Sc.
Submission Date:	30th March 2026

I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 30th March 2026 Abhijeet Abhay Sutar

Abstract

The Boundary Element Method (BEM) is a numerical technique that is well suited to model finite structures in an infinite exterior domain. However, the computational cost of the BEM scales as $\mathcal{O}(N^3)$, where N is the number of degrees of freedom, making it prohibitive for large-scale problems. The Fast Multipole Method (FMM) is an algorithm that can drastically reduce the computational cost to $\mathcal{O}(N^{\frac{3}{2}})$ or even $\mathcal{O}(N \log N)$, by decoupling far-field interactions through hierarchical spatial decomposition and truncated analytical basis expansions. This thesis presents the design, implementation, validation and performance analysis of a high-performance Fast Multipole Method (FMM) accelerated Boundary Element Method (BEM) solver.

AI use disclosure

This work has benefited from the use of AI-based language models in the following ways:

- Proofreading and grammar checking: I have used AI-based language models to proofread my writing and check for grammatical errors, specifically Claude's Sonnet 4.6
- Generate Latex equations: I have used AI-based language models to generate Latex equations for mathematical expressions, specifically Google's Gemini 3.1 Pro in its online interface.
- Generate plotting scripts: I have used AI-based language models to generate small plotting scripts for visualizing data, specifically Google's Gemini 3.1 Pro in its online interface, as well as, in VS Code with GitHub Copilot.

All scientific content, arguments, and structure were developed independently by the author.

Contents

1. Introduction

Simulation of wave propagation and scattering off of objects in infinite space can be challenging to solve with traditional numerical methods such as the Finite Element Method (FEM) or Finite Difference Method (FDM), which require discretization of the entire domain. They have to truncate the domain, and they need special treatment to handle the infinite domain, such as using Absorbing Boundary Conditions which apply artificial mathematical boundary conditions at these truncated boundaries to absorb outgoing waves and minimize unphysical reflections back into the domain. Another approach is to use Perfectly Matched Layer (PML)s which adds an artificial buffer zone around the domain that absorbs outgoing waves across all non-tangential angles. These methods can be computationally expensive, and due to numerical inaccuracies, they can introduce spurious reflections that can contaminate the solution.

The BEM takes a different approach by reformulating the problem as a boundary integral equation, which only requires discretization of the surface of the scatterer, rather than the entire domain. It satisfies the Sommerfeld radiation condition at infinity by construction. This, along with simpler meshing requirements, makes it eminently suitable for solving wave scattering problems in infinite domains. However, the reformulation causes spikes in error due to non-uniqueness of the solution at certain frequencies, known as the characteristic frequencies. While the Burton-Miller (BM) formulation can be used to mitigate this issue, it introduces additional singularities in the integral equation, which can be difficult to handle numerically. The BEM also results in a dense complex system of equations, which can't be solved using the Conjugate Gradient method, needing to be solved directly at a cost of $\mathcal{O}(N^3)$, where N is the number of degrees of freedom.

One approach to mitigate this issue comes from the unlikely fields of molecular dynamics and astrophysics, where the FMM was developed to efficiently compute long-range interactions between particles. The FMM splits the domain into a hierarchical tree structure, and approximates long-range interactions between non-adjacent nodes of the tree using multipole expansions. It also provides a way to perform the matrix-vector products needed to use iterative solvers without ever having to explicitly assemble the system matrix. This reduces the computational cost to $\mathcal{O}(N^{\frac{3}{2}})$ or even $\mathcal{O}(N \log N)$ if the tree is subdivided into multiple levels. This makes it possible to solve large scale problems with the BEM that would otherwise be intractable.

In this work, we present the design and implementation of a high-performance FMM-accelerated BEM solver for acoustic wave scattering problems. In Chapter 2 we discuss the mathematical formulation of the BEM and the BM approach. Chapter 3 focuses on the implementation of the FMM, specifically how to best subdivide the domain into a hierarchical tree structure for the FMM, and how to implement the various multipole expansions and translations needed for the FMM. In Chapter 4 we discuss the optimizations we make to improve performance on modern computing architectures, such as using SIMD instructions and optimizing memory access patterns. We detail the workaround we use to utilize `deal.II`'s native GMRES solver, which only supports real-valued systems, to solve the complex-valued system resulting from the BEM formulation. We present the validation of our implementation against analytical solutions, the control over the errors spikes achieved by the BM formulation, and analyze the performance of our solver against a traditional BEM approach in Chapter 5. In Chapter 6 we also discuss potential future directions for improving the solver and extending it to other types of wave scattering problems.

2. Mathematical Formulation of BEM

Before we discuss the implementation of the BEM for the Acoustic Helmholtz equation, we first need to discuss the Acoustic Wave equation. We can then show how the mathematical formulation of the Boundary Integral Equation (BIE) arises from the Acoustic Helmholtz equation. Finally, we will discuss the Burton-Miller formulation to address the issue of fictitious frequencies.

2.1. Acoustic Wave Equation

The propagation of acoustic waves in a homogeneous, inviscid fluid medium is governed by the linear wave equation for the acoustic pressure $p(\mathbf{x}, t)$:

$$\nabla^2 p(\mathbf{x}, t) - \frac{1}{c^2}\frac{\partial^2 p(\mathbf{x}, t)}{\partial t^2} = 0 \tag{2.1}$$

where $\mathbf{x}$ is the position vector, t is time, and c is the speed of sound in the medium.

To derive the Helmholtz equation, we consider a time-harmonic acoustic field. We assume the pressure can be represented as a complex function oscillating with angular frequency ω:

$$p(\mathbf{x}, t) = P(\mathbf{x})e^{-i\omega t} \tag{2.2}$$

where $P(\mathbf{x})$ is the spatial component of the pressure (the phasor) and $i = \sqrt{-1}$ is the imaginary unit.

Substituting Equation (2.2) into the wave equation (2.1):

$$\nabla^2 \left(P(\mathbf{x})e^{-i\omega t}\right) - \frac{1}{c^2}\frac{\partial^2}{\partial t^2}\left(P(\mathbf{x})e^{-i\omega t}\right) = 0 \tag{2.3}$$

Calculating the time derivatives:

$$\frac{\partial}{\partial t}\left(P(\mathbf{x})e^{-i\omega t}\right) = -i\omega P(\mathbf{x})e^{-i\omega t} \tag{2.4}$$

$$\frac{\partial^2}{\partial t^2}\left(P(\mathbf{x})e^{-i\omega t}\right) = (-i\omega)^2 P(\mathbf{x})e^{-i\omega t} = -\omega^2 P(\mathbf{x})e^{-i\omega t} \tag{2.5}$$

Substituting this back into the wave equation:

$$e^{-i\omega t}\nabla^2 P(\mathbf{x}) - \frac{1}{c^2}(-\omega^2 P(\mathbf{x})e^{-i\omega t}) = 0 \tag{2.6}$$

Dividing by the common time factor $e^{-i\omega t}$ (which is non-zero) and rearranging the terms:

$$\nabla^2 P(\mathbf{x}) + \frac{\omega^2}{c^2}P(\mathbf{x}) = 0 \tag{2.7}$$

Introducing the wavenumber $k = \omega/c$, we arrive at the homogeneous Helmholtz equation:

$$\nabla^2 P(\mathbf{x}) + k^2 P(\mathbf{x}) = 0 \tag{2.8}$$

Since we use the symbol ϕ to represent the acoustic potential in our BEM formulation, we can rewrite the Helmholtz equation as:

$$\nabla^2 \phi(\mathbf{x}) + k^2 \phi(\mathbf{x}) = 0 \tag{2.9}$$

2.2. Boundary Element Method

2.2.1. Boundary Integral Formulation

The Boundary Element Method (BEM) solves the Helmholtz equation by transforming it into an integral equation defined on the boundary of the domain. This transformation is typically achieved using Green's Second Identity.

Let Ω be the domain containing the fluid and Γ be its boundary. For two scalar fields ϕ and G in Ω, Green's Second Identity states [Wu00]:

$$\int_\Omega (\phi\nabla^2 G - G\nabla^2\phi)\, d\Omega = \int_\Gamma \left(\phi\frac{\partial G}{\partial n} - G\frac{\partial \phi}{\partial n}\right) d\Gamma \tag{2.10}$$

where n is the outward unit normal vector to the boundary.

We choose $\phi(\mathbf{x})$ to be the acoustic pressure, satisfying the Helmholtz equation derived in Equation (2.8):

$$\nabla^2 \phi(\mathbf{x}) + k^2 \phi(\mathbf{x}) = 0 \tag{2.11}$$

We choose $G(\mathbf{x}, \mathbf{x_0})$ to be the fundamental solution (Green's function) of the Helmholtz equation, which satisfies the inhomogeneous equation with a point source at $\mathbf{x_0}$:

$$\nabla^2 G(\mathbf{x}, \mathbf{x_0}) + k^2 G(\mathbf{x}, \mathbf{x_0}) = -\delta(\mathbf{x} - \mathbf{x_0}) \tag{2.12}$$

where δ is the Dirac delta function. For 3D acoustics, the free-space Green's function satisfying the Sommerfeld radiation condition is given by [Kir98]:

$$G(\mathbf{x}, \mathbf{x_0}) = \frac{e^{ikr}}{4\pi r} \quad \text{with} \quad r = |\mathbf{x} - \mathbf{x_0}| \tag{2.13}$$

Substituting these into Green's Second Identity for a source point $\mathbf{x_0}$ inside the domain Ω:

$$\phi(\mathbf{x_0}) = \int_{\Gamma} \left(G(\mathbf{x}, \mathbf{x_0}) \frac{\partial \phi}{\partial n}(\mathbf{x}) - \phi(\mathbf{x}) \frac{\partial G}{\partial n}(\mathbf{x}, \mathbf{x_0}) \right) d\Gamma(\mathbf{x}) \tag{2.14}$$

As the source point $\mathbf{x_0}$ is moved to the boundary Γ, the integrals become singular. Evaluating the singularity using a limiting process yields the Conventional Boundary Integral Equation (CBIE) [Mar08]:

$$c(\mathbf{x_0})\phi(\mathbf{x_0}) + \int_S \phi(\mathbf{x}) \frac{\partial G_k(\mathbf{x_0}, \mathbf{x})}{\partial n} dS(\mathbf{x}) = \int_S \frac{\partial \phi(\mathbf{x})}{\partial n} G_k(\mathbf{x_0}, \mathbf{x}) dS(\mathbf{x}) + \phi_{inc}(\mathbf{x_0}) \tag{2.15}$$

where $c(\mathbf{x_0})$, the integral free term, represents the geometric coefficient (solid angle). For a smooth boundary, $c(\mathbf{x_0}) = 0.5$.

2.2.2. Discretization

To solve the CBIE numerically, the boundary Γ is discretized into N boundary elements. The pressure ϕ and its normal derivative $v_s = \partial\phi/\partial n$ are approximated using shape functions.

Applying the collocation method at the nodes leads to a system of linear algebraic equations:

$$\mathbf{H}\boldsymbol{\phi} = \mathbf{G}\boldsymbol{v}_s \tag{2.16}$$

where $\mathbf{H}$ and $\mathbf{G}$ are the dense, nonsymmetric influence matrices resulting from the integration of the fundamental solution and its derivative, and $\boldsymbol{\phi}$ and $\boldsymbol{v}_s$ are vectors containing the nodal values of the pressure and normal velocity, respectively.

2.3. Burton-Miller Formulation

The Boundary Element Method (BEM) offers a highly elegant, mathematically rigorous, and computationally efficient alternative by reformulating the volumetric partial differential equation into a Boundary Integral Equation (BIE) mapped over the surface of the scattering or radiating body. This approach intrinsically satisfies the Sommerfeld radiation condition at infinity, thereby reducing the dimensionality of the problem by one—transforming a three-dimensional volumetric domain into a two-dimensional surface mesh, or a two-dimensional problem into a one-dimensional contour [Kir98].

However, as [BM71] showed, the standard BEM approach for exterior problems, gives rise to non-existence of unique solutions at wavenumbers coinciding with the eigenvalues or resonances of the corresponding interior Dirichlet problem.

At these critical frequencies, also sometime referred to as *fictitious or irregular frequencies*, the matrix system resulting from the discretization of the BIE becomes ill-conditioned, leading to numerical instability and a spike in the error of the computed solution.

It must be noted that the original exterior problem does have a unique solution at these frequencies and that the connection to the interior problem has no physical significance. The non-uniqueness arises solely from the formulation of the problem in terms of a BIE.[BM71]

There have been various approaches proposed in the literature to address this, such as the Combined Helmholtz Integral Equation Formulation (CHIEF) proposed by [Sch68]. The CHIEF method uses auxiliary collocation points within the interior to form an overdetermined system, which is then solved by a least-squares approach to eliminate the spurious solutions [CRA90]. However, the location of these auxiliary points is critical, and if they are placed on a nodal plane of an interior standing wave, the method can fail [LGB15]. By contrast, the formulation proposed by Burton and Miller [BM71] is a mathematically robust method that ensures the uniqueness of the solution across all frequencies.

2.3.1. The Hypersingular Boundary Integral Equation

The BM formulation combines the CBIE with its own normal derivative, which we will refer to as the Hypersingular Boundary Integral Equation (HBIE). We derive the HBIE by applying the gradient operator to the CBIE with respect to the collocation point $\mathbf{x_0}$, then taking the inner product with the local unit normal vector $\hat{\mathbf{n}}_\mathbf{0}$. The normal opertaor can be represented as $\frac{\partial}{\partial n_0} = \hat{\mathbf{n}}_\mathbf{0} \cdot \nabla_{\mathbf{x_0}}$. Applying this to the CBIE yields :

$$c(\mathbf{x_0})\frac{\partial \phi(\mathbf{x_0})}{\partial n_0} + \int_S \phi(\mathbf{x})\frac{\partial^2 G_k(\mathbf{x_0},\mathbf{x})}{\partial n \partial n_0} dS(\mathbf{x}) = \int_S \frac{\partial \phi(\mathbf{x})}{\partial n}\frac{\partial G_k(\mathbf{x_0},\mathbf{x})}{\partial n_0} dS(\mathbf{x}) + \frac{\partial \phi_{inc}(\mathbf{x_0})}{\partial n_0} \quad (2.17)$$

where n is the unit normal at the field point $\mathbf{x}$.

Like the CBIE suffers from non-uniqueness at the characteristic frequencies of the interior Dirichlet problem, the HBIE also suffers from non-uniqueness, but at the characteristic frequencies of the interior Neumann problem. As the eigenvalues of the interior Neumann problem do not coincide with those of the interior Dirichlet problem, the combination of the two equations ensures that their union has only one valid solution across all frequencies.

2.3.2. The Combined Equation and the Coupling parameter

The BM formulation is a linear combination of the CBIE and the HBIE, with the HBIE scaled by a complex coupling parameter η:

$$\text{CBIE} + \eta\text{HBIE} = 0 \tag{2.18}$$

A unique solution is guaranteed for (2.18) as long as η has a non-zero imaginary part. The theoretically optimal choice for this coupling parameter is $\eta = i/k$ in the high frequency limit. Terai[Ter] explains this by framing the Burton-Miller formulation as the boundary condition for a virtual interior acoustic problem, and relating the coupling parameter as $\eta = \frac{-1}{ikA}$, where A is specific admittance. By setting $A = 1$ to maximize the absorption, we can eliminate the possibility of internal resonances. At low frequencies, regularized variations such as $\eta = ik/\left(k^2 + \lambda\right)$in the limit $k \to 0$ to prevent matrix ill-conditioning [Mar16].

We can write the combine integral equation as:

$$c(\mathbf{x_0})\phi(\mathbf{x_0}) + \eta c(\mathbf{x_0})\frac{\partial\phi(\mathbf{x_0})}{\partial n_0} + \int_S \phi(\mathbf{x})\frac{\partial G_k(\mathbf{x_0},\mathbf{x})}{\partial n}dS(\mathbf{x}) + \eta\int_S \phi(\mathbf{x})\frac{\partial^2 G_k(\mathbf{x_0},\mathbf{x})}{\partial n\partial n_0}dS(\mathbf{x}) =$$
$$\int_S \frac{\partial\phi(\mathbf{x})}{\partial n}G_k(\mathbf{x_0},\mathbf{x})dS(\mathbf{x}) + \eta\int_S \frac{\partial\phi(\mathbf{x})}{\partial n}\frac{\partial G_k(\mathbf{x_0},\mathbf{x})}{\partial n_0}dS(\mathbf{x}) + \phi_{inc}(\mathbf{x_0} + \eta\frac{\partial\phi_{inc}(\mathbf{x_0})}{\partial n_0}) \tag{2.19}$$

While the BM formulation guarantees uniqueness, it comes at the cost of having to evaluate an integral term with a highly divergent kernel $\frac{\partial^2 G_k(\mathbf{x_0},\mathbf{x})}{\partial n\partial n_0}$. This integral will diverge and collapse the system, if not treated with specialized analytical techniques.

2.3.3. Characterizing the Boundary Integral Singularities

To understand the mathematics of the hypersingularity, and subsequently the techniques to regularize it, we first need to explicitly derive the derivatives of the Green's function and characterize their behavior. Consider the green's equation in 3D:

$$G_k = \frac{e^{ikr}}{4\pi r}$$

and let the distance vector be $\tilde{\mathbf{r}} = \mathbf{x} - \mathbf{x_0}$, with scalar distance $r = |\tilde{\mathbf{r}}|$. We can then write the gradient of the distance with respect to the collocation point as $\nabla_{\mathbf{x_0}} r = -\frac{\tilde{\mathbf{r}}}{r}$, and with respect to the field point as $\nabla_{\mathbf{x}} r = \frac{\tilde{\mathbf{r}}}{r}$.

First Normal Derivative

The first normal derivative of the Green's function with respect to normal n atthe field point $\mathbf{x}$ is then given by:

$$\frac{\partial G_k}{\partial n} = n \cdot \nabla_x \left(\frac{e^{ikr}}{4\pi r} \right) = \frac{e^{ikr}}{4\pi r^2}(ikr - 1)\frac{n \cdot (x - x_0)}{r} \tag{2.20}$$

As $r \to 0$, i.e., when we get close to the collocation point, the term $e^{ikr} \to 1$. The functional behavior is dominated by the $1/r^2$ term. However, for a smooth surface the, the normal vector n becomes perpendicular to the distance vector as the points merge. This means that the term $n \cdot (x - x_0)$ approaches zero at the rate of $\mathcal{O}(r^2)$, and this reduces the apparent $\mathcal{O}(1/r^2)$ singularity to a weak $\mathcal{O}(1/r)$ singularity. Considering the fact that our mesh elements are locally smooth, this applies to our case as well.

Second Normal Derivative (Hypersingularity)

Applying the normal derivative again to (2.20) with respect to the normal direction n_0 at the collocation point $\mathbf{x_0}$, we get:

$$\frac{\partial^2 G_k}{\partial n \partial n_0} = \frac{1}{4\pi} \left[\frac{n_0 \cdot n}{r^3}(1 - ikr)e^{ikr} - \frac{(n \cdot (x - x_0))(n_0 \cdot (x - x_0))}{r^5}(3 - 3ikr - k^2r^2)e^{ikr} \right] \tag{2.21}$$

To characterize the behavior of this kernel, we use the Taylor expansion of the exponential term e^{ikr} for infinitesimally small values of r.

$$e^{ikr} = 1 + ikr - \frac{1}{2}k^2r^2 - \frac{i}{6}k^3r^3 + \mathcal{O}(r^4) \tag{2.22}$$

Using this expansion, the first term in (2.21) expands to:

$$(1 - ikr)e^{ikr} \approx 1 + \frac{1}{2}k^2r^2 + \mathcal{O}(r^3) \tag{2.23}$$

and the second term expands to:

$$(3 - 3ikr - k^2r^2)e^{ikr} \approx 3 + \frac{1}{2}k^2r^2 + \mathcal{O}(r^3) \tag{2.24}$$

Substituting these expansions back into (2.21), we get

$$\frac{\partial^2 G_k}{\partial n \partial n_0} = \frac{1}{4\pi} \left[\frac{n_0 \cdot n}{r^3} \left(1 + \frac{1}{2}k^2r^2 \right) - \frac{(n \cdot \vec{r})(n_0 \cdot \vec{r})}{r^5} \left(3 + \frac{1}{2}k^2r^2 \right) \right] + \mathcal{O}(1) \tag{2.25}$$

[SK23]

Table 2.1.: Singularity Behavior of the Green's Function and its Derivatives

Kernel Type	**Associated Equation**	**Functional Form**	**Singularity Order (3D)**	**Integration Requirement**
Weakly Singular	CBIE (RHS)	G_k	$\mathcal{O}(1/r)$	Standard Quadrature
Strongly Singular	CBIE (LHS)	$\partial G_k/\partial n$	$\mathcal{O}(1/r^2)$	CPV
Hypersingular	HBIE (LHS)	$\partial^2 G_k/\partial n \partial n_0$	$\mathcal{O}(1/r^3)$	HFP

This reveals that the singularity is fundamentally dominated by the $1/r^3$ term. Integrals diverging as $\mathcal{O}(1/r^3)$ are strictly hypersingular, and cannot be evaluated using Cauchy Principal Value theorems. Standard numerical quadratures like the Gauss-Legendre quadrature will fail to converge for such integrals without applying analytical regularization.

2.3.4. Flat Panel Simplification: Geometric Reduction

If we use flat panels to discretize the boundary, whether flat triangles or flat quadrilaterals, we can leverage a geometric simplification that allows us to reduce mathematical complexity of the BM formulation. This arises from the fact that the normal vector will be constant across a flat element, and the distance vector will be always be perpendicular to the normal vector.

Orthogonality and Parallelism Constraints

When the integration point is close enough to the collocation point for the singularity to manifest, both points are located on the same flat panel. This geometry establishes two critical constraints:

1. Parallel Normal Vectors: The normal the the collocation point n_0 and the normal at the field point n are identical and parallel.

$$n(x) = n(x_0) = n_0 \Longrightarrow n_0 \cdot n = 1$$

2. Orthogonality of the Distance Vector: The distance vector $\vec{r} = x - x_0$ is always perpendicular to the normal vector, as both points lie on the same plane.

$$n \cdot (x - x_0) = 0 \text{ and similarly} n_0 \cdot (x - x_0) = 0$$

Hypersingularity Kernel Collapse

Using these constraints in the second normal derivative of the Green's function (2.21), we can see that the second term goes to zero:

$$\frac{\partial^2 G_k}{\partial n \partial n_0} = \frac{1}{4\pi} \left[\frac{1}{r^3}(1 - ikr)e^{ikr} - \frac{(0)(0)}{r^5}(3 - 3ikr - k^2r^2)e^{ikr} \right] \tag{2.26}$$

The second term, which captured the curvature of the surface, collapses to zero. The hypersingular kernel collapses down to:

$$\left. \frac{\partial^2 G_k}{\partial n \partial n_0} \right|_{\text{on a single flat panel}} = \frac{1}{4\pi} \left[\frac{1}{r^3}(1 - ikr)e^{ikr} \right] \tag{2.27}$$

This particular simplification means that we don't have to compute the extra terms associated with the normals nd local tangents.

Decoupling Static and Dynamic Components

While our flat panel geometry allows us to collapse the functional form of the kernel, the expression $\frac{e^{ikr}}{r^3}(1 - ikr)$ is still hypersingular. To compute the term numerically, we split the integrand into a purely divergent, static singularity, and a regularized, weakly singular dynamic component [SK23]. Looking at the Taylor expansion of the kernel after the collapse:

$$\frac{e^{ikr}}{r^3}(1 - ikr) = \frac{1}{r^3} \left[1 + \frac{1}{2}k^2r^2 + \mathcal{O}(r^3) \right] \tag{2.28}$$

$$= \frac{1}{r^3} + \frac{1}{2r}k^2 + \mathcal{O}(1) \tag{2.29}$$

This expansion tells us something quite important about thekernel's frequency dependence. The dynamic term that depends on the wavenumber k is only weakly singular, of the order of $\mathcal{O}(1/r)$. The remaining term, which is purely static and real, is the hypersingular term $1/r^3$.

Sun and Klaseboer [SK23] demonstrate that this isolated static term $1/r^3$ is the same as the hypersingular kernel of the Laplace equation, which we can consider as a special case of the Helmholtz equation where $k = 0$. We can demonstrate this equivalence by taking the second normal derivative of the fundamental solution to the Laplace equation $G_0 = \frac{1}{4\pi r}$, which is given by:

$$\frac{\partial^2 G_0}{\partial n \partial n_0} = \frac{1}{4\pi} \left[\frac{n_0 \cdot n}{r^3} - \frac{3(n \cdot (x - x_0))(n_0 \cdot (x - x_0))}{r^5} \right] \tag{2.30}$$

Applying the flat panel orthogonality and parallelism constraints again, we can see that the second term goes to zero, and the kernel collapses to:

$$\left. \frac{\partial^2 G_0}{\partial n \partial n_0} \right|_{\text{flat panel}} = \frac{1}{4\pi} \left[\frac{1}{r^3} \right] \tag{2.31}$$

This lets us break down the acoustic Helmholtz hypersingular kernel perfectly into the static Laplace hypersingular kernel, and a regularized dynamic remainder:

$$\left. \frac{\partial^2 G_k}{\partial n \partial n_0} \right|_{\text{flat panel}} = \left. \frac{\partial^2 G_0}{\partial n \partial n_0} \right|_{\text{flat panel}} + \frac{1}{4\pi} \left[\frac{e^{ikr}(1 - ikr) - 1}{r^3} \right] \tag{2.32}$$

The isolated term in the brackets is completely regularized, and its limit as $r \to 0$ evaluates to $\frac{k^2}{2r}$. Since this term is only weakly singular, we can use standard numerical quadratures, such as the Gaussian quadrature, to integrate it over the flat panel without divergence.

Through this geometric simplification and asymptotic decoupling, the highly complex Burton-Miller implementation problem on flat panels is reduced to a singular, specific mathematical goal: evaluating the finite part of the static Laplace hypersingular integral $\int_S \frac{1}{r^3} dS$ over a planar polygonal element. In our code we integrate it with a higher order quadrature to ensure accuracy.

3. The Fast Multipole Method Framework

The BEM lends itself naturally to modelling acoustic wave propagation. Transforming the volumetric governing differential equations into boundary integral equations allows finite structures to be treated in an infinite exterior domain without having to truncate the domain, thereby avoiding the need for special boundary conditions. However, the required mathematical transformation of the problem yields a full, complex, non-Hermitian matrix [CL07]. As the degrees of freedom increase to model larger problem, the memory requirements to hold the large full matrix and the cost to compute the matrix coefficients themselves, scale as $\mathcal{O}(N^2)$, and the computational cost of solving the resulting linear system using direct solvers scales as $\mathcal{O}(N^3)$, where N is the number of degrees of freedom [Liu09]. This makes the BEM computationally prohibitive for large-scale problems.

To overcome this computational bottleneck, we must employ a radically different numerical strategy. This chapter details the algorithmic design and the mathematical formulation of the FMM framework. The FMM decouples far-field interactions through hierarchical spatial decomposition, truncated analytical basis expansions, and the application of complex mathematical translation operators [CRW93].

This chapter focuses on the mathematical and algorithmic design of the FMM implemented as a part of this thesis. It details the division of the mesh space into a topological tree data structure, the mathematics of the multi-stage multipole translations, and the implementation of near-field interactions.

3.1. Fast Multipole Method

The FMM was introduced in 1987 by Greengard and Rokhlin [GR87] to accelerate large-scale particle simulations involving long-range pairwise potential interactions. Later, it was extended to other problems, including the BEM for Acoustic Scattering [Rok90]. The FMM handles short-range interactions with direct computation, and uses analytical multipole expansions to compute potentials for long-range interactions. By providing a method for the rapid computation of the matrix-vector product between the system matrix any arbitrary vector, the FMM enables the use of iterative solvers for solving the linear systems arising from the BEM discretization, reducing the computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N^{\frac{3}{2}})$ or even $\mathcal{O}(N \log N)$ for a multilevel implementation [CRW93].

3.1.1. The FMM algorithm

The FMM follows these basic steps:

Step 1. **Discretization** Any problem begins with discretizing the boundary, as is done for conventional BEM. We use constant elements.

Step 2. **Initializing the Octree** For a 3D problem, we consider a cube that contains the entire geometry. This cube is repeatedly subdivided into 8 smaller cubes, called its child nodes, creating an Octree structure. The subdivision continues until the number of elements in a cube falls below a defined threshold.

Step 3. **Upward Pass** Beginning from the bottom of the tree, at the deepest level called the leaf nodes, we compute the moments of each node, tracing the tree structure upwards till we are two levels below the root node. Using the M2M translation operator, the moments of the child nodes are translated and aggregated to the parent node.

Step 4. **Downward Pass** Starting from the second level of the tree, we compute the local expansions for each node. This consists of contributions from the current node's interaction list and from far-off nodes. The interaction list nodes are those not directly adjacent to the current node, but whose parent node is adjacent to the current node's parent. The contribution from the interaction list nodes is computed using the M2L translation operator, while the contribution from the far-off nodes is computed using the L2L translation operator for the parent node, with the expansion point being shifted from the centroid of the parent node to the centroid of the current node. At level 2, the nodes only have contributions from the interaction list nodes. Since there are no far-off nodes at this level, we use the M2L operator to get the local expansion's coefficients.

Step 5. **Evaluating of the Integrals** The contributions of all the collocation points in the current node and thed nodes directly adjacent to it are computed using direct BEM. The contributions of all nodes in the interaction list and the far-off are computed using the local expansion coefficients of the current node and shifting the expansion point to the collocation point using the Local-to-Particle (L2P) translation operator.

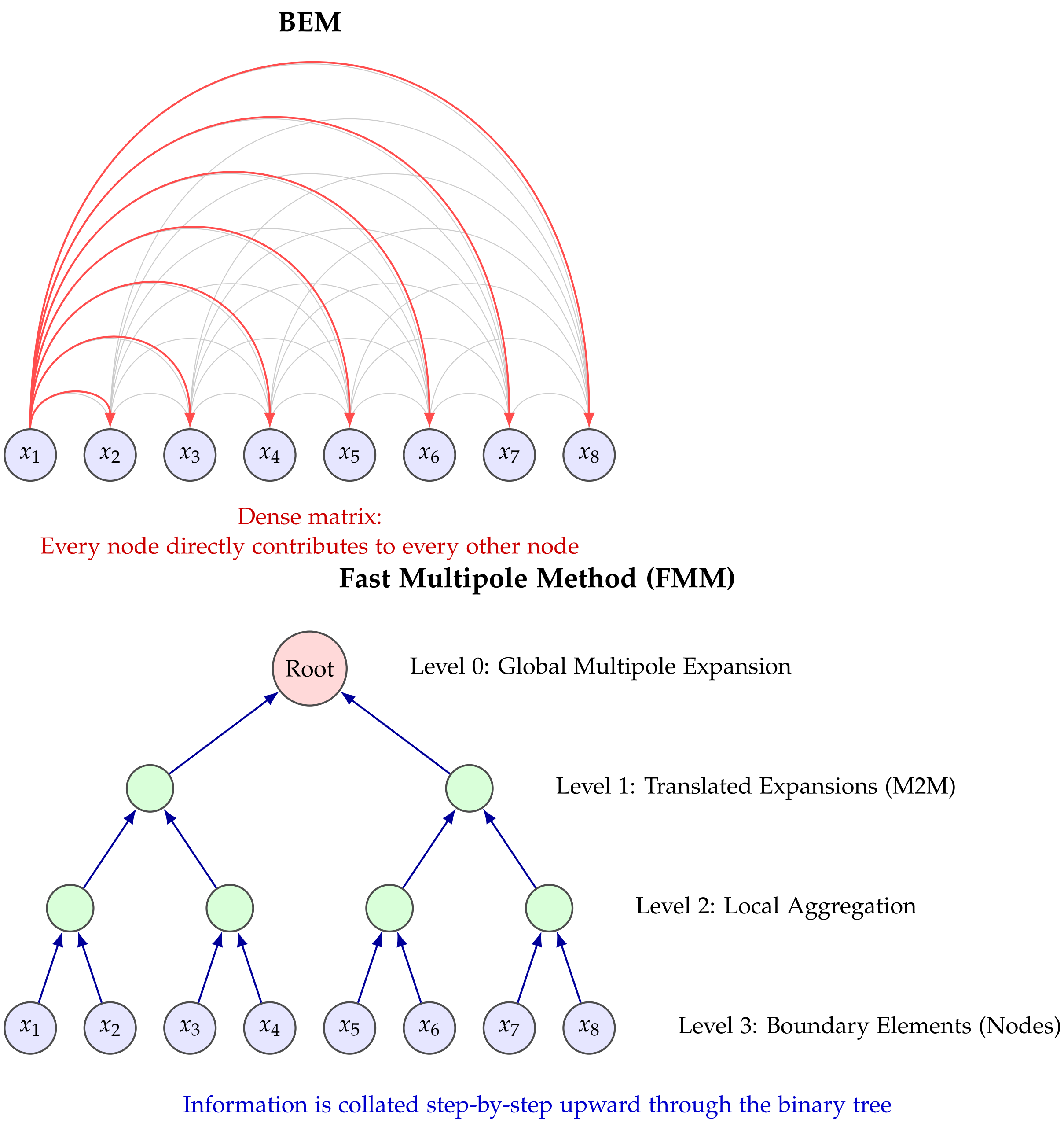


Figure 3.1.: A visual comparison of the computational complexity of the basic BEM and the FMM. In BEM, every node directly contributes to every other node, resulting in a dense matrix. The FMM allows far-field interactions to be approximated through multipole expansions, significantly reducing the number of direct interactions.

Step 6. Iterative Solver We update the unknown vector, in this case the `phi` vector in the system of equations $\mathbf{A}\boldsymbol{\phi} = \mathbf{b}$, using an iterative solver, such as GMRES, and repeat from steps 3 until convergence is achieved.

3.2. Octree

The spatial domain can be subdivided into a hierarchical tree structure in many ways. For 3D problems, the most common approach is to use an Octree, which offers many opportunities to optimize algorithmic efficiency. An Octree is a tree data structure where each node is bisected along the Cartesian x, y, and z axes, producing 8 equally sized child nodes, and these child nodes are then further subdivided into 8 nodes until a certain threshold is reached.

3.2.1. Nodes of the Octree

We define an `FMMNode` data structure to represent each node in the Octree, which contains the following information:

Node geometry: The struct keeps track of the node's bounding box, its center, and its radius. It also stores the integer grid coordinates that represent the node's position in the Octree.

Tree Hierarchy: Each node is assigned a node index, which serves as the node's unique identifier within a global list of all the nodes. The node also stores its tree level and the index of its parent node. Most importantly, it stores the indices of its 8 child nodes, which are used to navigate the tree.

Mesh and Interaction data: The node stores an iterator for the particles contained within it. It maintains a neighbor list that contains the indices of neighboring nodes. These interactions must be computed directly. It also stores a list of far-field nodes, which are sufficiently far away to be approximated by multipole expansions.

Mathematical expansion data: The node stores `Pm` which is the order used for the mutlitpole expansion, and the number of coefficients required based on the order. It also stores the multipole and local expansion coefficients, which are used to approximate the potential and forces from distant nodes.

FMM Translation Operators: The node stores the necessary translation operators for the FMM algorithm.

1. Particle-to-Multipole (P2M) Matrix: Translates raw particles (mesh sources) to a node's Multipole expansion.
2. M2M Matrix: Translates a child's Multipole expansion up to its parent's Multipole expansion.
3. M2L Matrices: Converts a distant node's Multipole expansion into this node's Local expansion.
4. L2L Matrix: Translates a parent's Local expansion down to its child's Local expansion.
5. L2P Matrix: Evaluates the Local expansion directly onto the target particles (mesh cells).

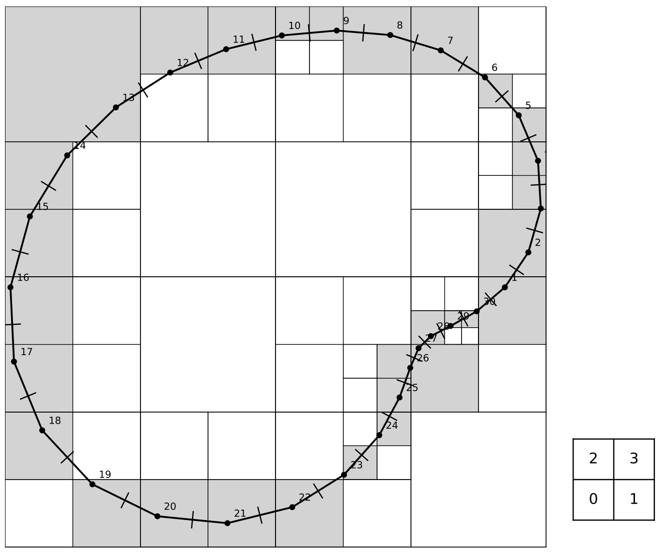


Figure 3.2.: An example of a 2D domain with a 1D boundary being split up into a Quadtree. The small square on the side shows the numbering scheme for the child cells of any given parent cell [LN06]

3.2.2. Splitting the domain to make an Octree

The underlying spatial data-structue sets the stage for the computational efficiency of the FMM algorithm. It dictates how the source-target interactions are categorized as either near-field, needing direct computation, or far-field, and thus suitable for approximation via multipole expansions. An adaptive, heirarchical Octree allows for a systematic way to manage these interactions. The Octree is constructed by a top-down recursive subdivision of the spatial domain.

We start by defining the root node as a cube that encompasses the entire geometry. We iterate over the vertices of all the mesh elements to find the minimum and maximum coordinates. We then add a 1% padding to the bounding box as a safety margin to ensure that all elements are contained within the root node, and to avoid edge cases where elements lie exactly on the boundary of the node. This safety margin can help guard against floating point precision issues occurring precisely at the boundary, which could lead to elements on or near the boundary being misclassified as outside the node. This root node is then subdivided into 8 child nodes, and the elements are assigned to the child nodes based on their centroids. This is repeated recursively for each child node until the number of elements left in a node are less than a predefined threshold, or a set maximum depth is reached, at which point the node is designated as a leaf node. This threshold is a critical parameter that can be manually set at runtime. When subdividing the domain for a multi-object mesh, we can see that there will be empty interstitial space. The tree adapts to this by pruning empty branches, that is bracnhes the do not contain any boundary elements. Each instantiated `FMMNode` is assigned a unique spatial index, and information like its depth, the index of its parent node, its geometric coordinates, and the indices of its child nodes are stored in the node's data structure. This information is crucial for the FMM tree traversal.

The core of the FMM logic revolves around the interaction lists of each node generated by the `build_interaction_lists` method after the Octree geometry is constructed. This method operates over the Tree as follows:

1. **Level 0 (Root Node):** Considering the root node has no neighbors, its far-field list remain empty, and its neighbor list consists only of itself, as it interacts exclusively with itself.

2. **Level 1 (First level of child nodes):** These intial eight child nodes of the root node are neighbors to each other, and thus their neighbor lists consist of each other. From this it follows that they have no far-field interactions, and thus their far-field lists also remain empty.

3. **Level 2 and Deeper:** To build the neighbor and far-field lists for nodes at level 2 and deeper, we first identify the parent node, and then iterate through the parent's neighbor list. For each neighboring node of the parent, we check if its children, colloquially referred to as the "cousin" nodes of the current node, are adjacent to the current node or not. If they are, we add them to the neighbor list.

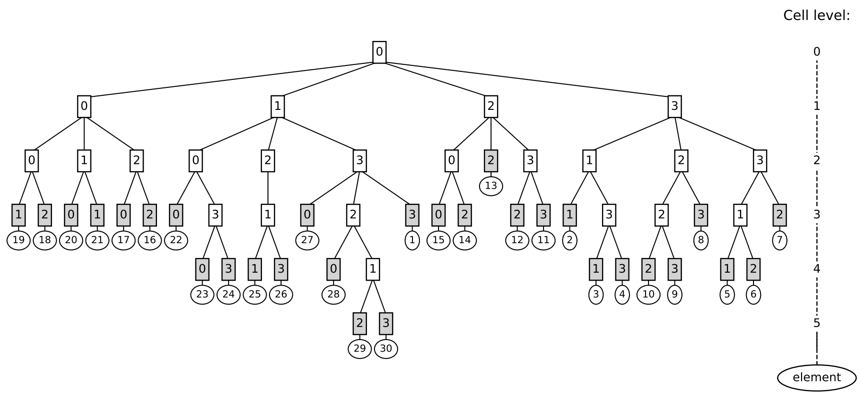


Figure 3.3.: Quadtree structure corresponding to the subdivided domain in Figure 3.2. The line at the right shows the depth of the tree. The leaf nodes are colored in gray andthe circle beneath them shows the ID of the mesh element they contain. The non-leaf nodes are white rectangles. All nodes are labeled with their local ID in their parent node as per the scheme shown in the small square in Figure 3.2. [LN06]

Space-Filling Curves and Morton Encoding

We map the geometric centers of the boundary elements from three-dimensional space into a one-dimensional array using a space-filling curve, specifically, the Morton Z-curve. Morton encoding converts the 3D coordinates of each mesh element into a single integer by interleaving the bits of the binary representations of the x, y, and z coordinates, generating that element's Morton key. This ordering assigns numerically adjacent Morton keys to spatially adjacent elements. This allows us to sort the elements into a linearly ordered array, where adjacent elements get clustered together, improving cache locality. This cache locality helps reduce the overhead of assigning the elements to the Octree's nodes. While the Morton Z-order curve may not be the optimum space-filling curve for 3D problems, its simplicity of implementation via bitwise operations lends itself to efficient execution [MB15].

3.2.3. Admissibiltiy Criterion and the Interaction Lists

We use a strict geometric admissibility criterion to determine whether a candidate "cousin" node goes into the target node's `neighbor_list` or the `interaction_list`. We use the Chebyshev distance, which is the maximum of the absolute differences of their Cartesian coordinates, to evaluate spatial separation. Given a pair of target node and candidate cousin node at the same heirarchical level, we extract their relative integer grid coordinates `grid_x, grid_y, grid_z`, and compute the spatial dimension differences: dx, dy, and dz. The admission criterion is then evaluated as follows:

- **Near-field interaction (The neighbor list):** The maximum orthogaonal separation is calculated as $\max(|dx|, |dy|, |dz|)$. If this is less than or equal to 1, the candidate cousin node share at least a vertex, edge or even a face. This means that the nodes are geometrically adjacent, and thus their interactions must be computed directly. Therefore, the candidate cousin node is added to the target node's `neighbor_list`. During the matrix-vector product stage of the FMM, these neighbor nodes will be evaluated with exact Particle-to-Particle (P2P) analytical integration.
- **Far-field interaction (The interaction list):** If the maximum orthogonal separation is greater than 1, thenit means that the candidate cousin node is at least separated from the target node by the width of one cubic node. Therefore, this candidate node being sufficiently far away, it is added to the target node's `interaction_list`. During the matrix-vector product stage of the FMM, the acoustic influence of these nodes will be transferred to the target node using the M2L translation operator.

There is one topological edge case that we must take care of. This arises when the parent's neighbor node is a coarser leaf node. What this means, is that the parent node's neighbor node contain some elements of the boundary, but was not subdivided further because the number of elements it contained was less than the threshold, thus terminating at a shallower heirarchical depth. Since the M2L operators are formulated for nodes at the same heirarchical level, we cannot directly use the M2L operator to compute the interaction between the target node and this coarser neighbor node. To maintain numerical stability, we inject the parent's coarser leaf neighbor into the target node's `neighbor_list` and compute the interaction directly with P2P integration despite their potential separation.

3.2.4. Wiscombe's Criterion for the Truncation Order

The theoretical basis of the FMM lies on the infinite series expansion of the Green's function. However, in practice, we must truncate this expansion to a finite number of terms, deonted as P. The choice of P is a critical trade-off between accuracy and computational cost. The number of complex-valued coefficient required to represent the acoustic field scaler quadratically with P as $N_{coeff} = (P+1)^2$ for a 3D problem.

Unlike the Laplace equation, where the truncation error decays geometrically with distance, the Helmholtz equation displays oscillatory phase behaviours that complicate the choice of the truncation order. If the physical size of a node, in this case its diameter is large relative to the acoustic wavelength, then the expansion needs to contain sufficient terms to capture the spatial oscillations, otherswise, the translated field will experience severe aliasing and phase destruction. To ensure accuracy across the whol domain, we calculate a uniform truncation order for all the nodes in the tree based on the largest node diameter and the wavenumber, using Wiscombe's criterion, which is formalized in the context of the FMM as the Excess Bandwidth Formula by Song and Chew [Che01]. The algorithm of the methoda that determines the truncation order is as follows:

1. **Initilize Precision:** The function accepts the acoustic wavenumber k, and a user define tolerenace ϵ. This tolerance is converted to a precision metric d_0 using the formula $d_0 = -\log_{10}(\epsilon)$, which represents the number of significant digits required.

2. **Calculate the Size Parameter:** A non-dimensional size parameter, kD, is calculated where D is the absolute maximum diameter among all the leaf nodes. kD represents the physical diameter of the node normalized by the acoustic wavenumber.

3. **Regime Bifurcation:**
   - **Low-Frequency Regime:** If $kD < 1.0$, the node is small relative to the wavelength, the expansion behave like the static Laplacian potential. The truncation order can then be appointed by a stable low-frequency heuristic:

     $$P = [kD + 5]$$

     .
   - **High-Frequency Regime:** If $kD \geq 1.0$, the node spans one or multiple wavelengths, in which case we need to apply the Excess Bandwidth Formula. It scales the required terms linearly with the size parameter kD, while adding a non-linear excess bandwidth term driven by the desired accuracy d_0. The order is calculated as:

     $$P \approx [kD + 1.8 \cdot d_0^{2/3} \cdot (kD)^{1/3}]$$

Subsequently, a uniform truncation order P is assigned identically to all the nodes in the tree to ensure that all the translation matrices are perfectly square and dimensionally compatible.

3.3. Expansion Bases and Key building blocks of the FMM

Before we can derive the translation operators, we need to define the expansion bases used for the multipole and local expansions. The translations of the exterior acoustic Helmholtz equation requires the use of spherical coordinate bases, specifically spherical harmonics for angular resolution, and spherical Bessel and Hankel functions for radial propogation.

3.3.1. Fundmental Solution for the Helmholtz

The free-sapce Green's function for the 3D Helmholtz equation which charactesrizes the acoustic pressure at a target point $\mathbf{x}$ due to a unit point source at at $\mathbf{x_0}$ is given by:

$$G(\mathbf{x}, \mathbf{x_0}) = \frac{e^{ik|\mathbf{x}-\mathbf{x_0}|}}{4\pi|\mathbf{x}-\mathbf{x_0}|} \tag{3.1}$$

For the FMM, the Cartesian coordinates (x, y, z) taken rlative to the center of the octree node are transformed to spherical coordinates (r, θ, ϕ). The acoustic field is then represented as a linear combination of spherical wave functions, mapping the global Cartesian based interactions to a local spherical coordinate system.

3.3.2. Spherical Harmonics and Legendre Polynomials

Spherical Harmonics, denoted as $Y_n^m(\theta, \phi)$, are a set of orthogonal functions defined on the surface of a unit sphere, and act as the angular portion of the general solution to the vector Helmhholtz equation The integer $n \geq 0$ is the degree of the harmonic, while the integer m represents the order.

Our FMM framework uses an orthonomal normalization convention that rigorously ensures that the surface integral of the squared magnitude of the harmonic over the unit sphere evaluates to exactly unity. Uder this specific normalization regime, we can define the spherical harmonic in terms of the associated Legendre polynomials $P_n^m(\cos\theta)$ as:

$$Y_n^m(\theta, \phi) = \sqrt{\frac{(2n+1)(n-m)!}{4\pi(n+m)!}} P_n^m(\cos\theta) e^{im\phi} \tag{3.2}$$

The evaluation of the associated Legendre polynomials is performed using the numerically stable 3-term recurrence relation, maintaining bounding limits to prevent floating point underflow at high expansion degrees. Setting the evaluation parameter $x = \cos\theta$, the generalized non-diagonl terms $P_n^m(x)$ can be constructed as:

$$P_n^m(x) = \frac{x(2n-1)xP_{n-1}^m(x) - (n+m-1)P_{n-2}^m(x)}{n-m} \tag{3.3}$$

3.3.3. Spherical Bessel and Hankel Functions

The radial propagation of the acoustic wave is governed by the spherical Bessel differential equation. Depending on the topology of the translation pass, we need two distinct classes of solutions:

1. **Spherical Bessel functions of the First Kind ($j_n(kr)$):** These are regular (finite) at the origin $r = 0$. They are used to represent standing waves or incoming acoustic fields. In the context of the FMM, they are used for the M2M upward shifts and the L2L downward shifts, where these operations occur strictly with the continuous domain of an octree node, and the field is nonsingular.

2. **Spherical Hankel functions of the First Kind ($h_n^{(1)}(kr)$):** These model outward-propagating, radiating waves that satisfy the Sommerfeld radiation condition. They have an intrinsic singularity at the origin, diverging towards complex infinity as $r \to 0$. They are used to the M2L translations in the Horizontal passes, where the model the radiation of scattered energy across the far-field gap to a separated target cluster.

3.3.4. Wigner 3-j Symbols and Racah's Formula

The mathematical translation of spherical wave expansions requires integrating the product of multiple spherical harmonics. This process relies on the Wigner 3-j symbols, which arise from the quantum mechanical theory of angular momentum coupling [GD04].

The Wigner 3-j symbols, denoted as $\begin{pmatrix} j_1 & j_2 & j_3 \\ m_1 & m_2 & m_3 \end{pmatrix}$, dictate the geometric probability amplitude of coupling three distinct spherical angular states. We evaluate this symbols using the exact, analytical Racah formula. To prevent unnecessary computations, the algorithm first enforces strict selection rules, and returns 0.0 if any of the following physical limits are violated:

1. **Magnetic Conservation:** The sum of the order indices must be strictly zero, i.e., $m_1 + m_2 + m_3 = 0$.

2. **Triangule Inequality:** The degrees must form a closed vector triangle, i.e., $|j_1 - j_2| \leq j_3 \leq j_1 + j_2$.

3. **Order Bounds:** the absolute value of any order must not exceed its associated degree: $|m_i| \leq j_i$.

If the selection rules are satisfied, the algorithm proceeds to compute the Wigner 3-j symbol using Racah's summation over an intermetdiate variable k.

Perfomance Optimization

Because the Wigner 3-j symbols are called from deep within the deepest nested loops of the translation matrix assembly, invoking standard library gamma functions to evaluate the factorials in Racah's formula would be computationally prohibitive. To get peak performance, we eliminate runtime factorial evaluations almost entirely by generating a highly optimized static lookup table during the solver's initialization phase.

3.3.5. Gaunt Coefficients

The Gaunt coefficient G, defined as $G(l_1, m_1; l_2, m_2; l_3, m_3)$, is the core of all the translation operators. It represents the exact analytical integral of the product of three distinct spherical harmonics over the surface of a unit sphere:

$$G(l_1, m_1; l_2, m_2; l_3, m_3) = \int_{S^2} Y_{l_1}^{m_1}(\Omega) Y_{l_2}^{m_2}(\Omega) Y_{l_3}^{m_3}(\Omega) d\Omega \tag{3.4}$$

We evaluate the Gaunt coefficients using the computationally efficient Wigner 3-j symbols. Apart from the selection rules inherited from the Wigner 3-j symbols, the Gaunt integral enforces a rigid Parity Rule. The integral vanishes entirely if the sum of the degrees $l_1 + l_2 + l_3$ is an odd integer.

Assumiing the selection rules are satisfied, the Gaunt coefficient is calculated as the product of two Wigner 3-j symbols scaled by a geometric normalization prefactor the arises from the orthonormal normalization convention of the spherical harmonics:

$$G = \sqrt{\frac{(2l_1+1)(2l_2+1)(2l_3+1)}{4\pi}} \begin{pmatrix} l_1 & l_2 & l_3 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} l_1 & l_2 & l_3 \\ m_1 & m_2 & m_3 \end{pmatrix} \tag{3.5}$$

3.4. Building the Translation Matrices

With the foundational mathematics now defined, we can now construct the translation matrices that are the core of the FMM algorithm. These operators map the coefficients of an acoustic series expansion centered at one spatial location to an equivalent, physically identical series expansion centered at a new geometric origin. Using the broadband formulations described by [GD09], the FMM framework leverages addition theorems to generate structured dense matrices for all the necessary translations, including

M2M, M2L, and L2L translations. These matrices are precomputed and stored for efficient reuse during the algorithm's execution, significantly reducing the immense computational overhead associated with on-the-fly calculations.

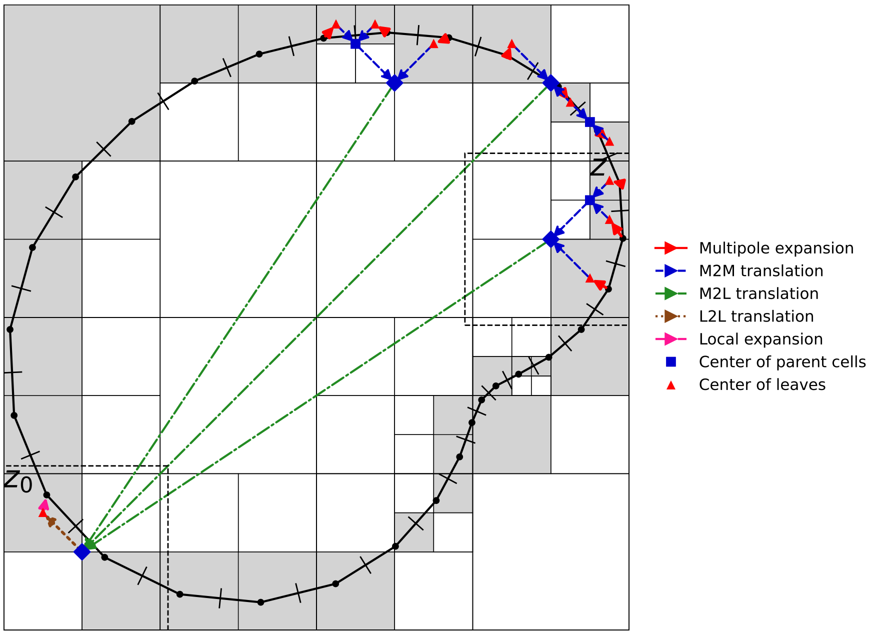
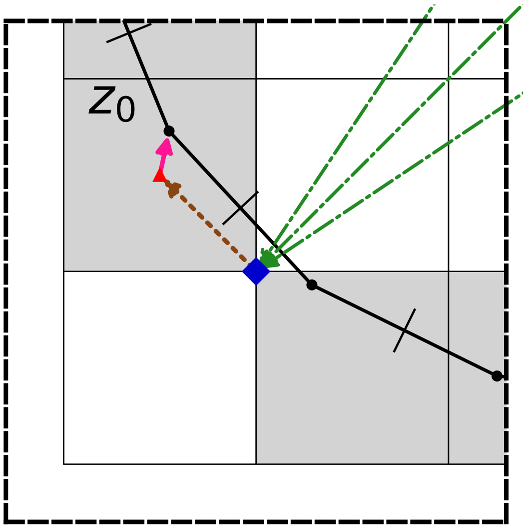


Figure 3.4.: A simplified representation of the FMM algorithm's main steps. This follows the same 2D system from 3.2 and 3.3. We show some of the far-field interactions and how they are computed. The green arrows represent the M2L translations, the blue arrows represent the M2M translations, and the red arrows represent the Multipole Expansions. The brown arrows represent the L2L translations, and the orange arrows represent the L2P local expansions. [LN06]

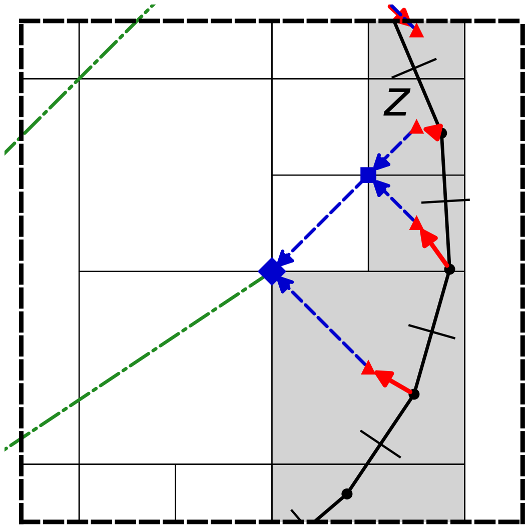
3.4.1. Point-to-Multipole (P2M) Translations

The P2M translation is the first step in the upward pass of the FMM algorithm. It calculates the influence of the boundary elements within a leaf node and projects them into multipole coefficients relative to the geometric center of the leaf node.

The Collocation method for the BEM that we use assigns both potential and flux distributions to boundary elements. This cause the mesh surfaces to act as continuous distributions of acoustic dipole. The multipole coefficients M_n^m for an expansion order P are accumulated in a flattened one-dimensional array of size $(P+1)^2$, where the



(a) We can see how the incoming information from the M2L interactions is translated down to the leaf node, and the L2P local expansion translates it onto the boundary element.



(b) The upwards translation process, where the multipole expansion of the boundary element is translated up to the leaf node's center, and then up to the parent node's center with the M2M translation, before being translated to the far-field node's center with the M2L translation.

Figure 3.5.: Close up views of the translation process for the target and source nodes from 3.4 [LN06]

memory index is calculated optimally as $n^2 + n + m$, running linearly from $n = 0...P$ and $m = -n...n$.

We implement two methods compute the source projection, the first is an Analytical method which provides mathematically exact derivatives, and a FDM as fallback for numerical stability near singularities.

The Analytical P2M

The exact mathematical basis function that represents the outgoing acoustic expansion is given by the product of the regular spherical Bessel function and the complex conjugate of the spherical harmonic function like so:

$$f_n^m(r, \theta, \phi) = j_n(kr)\overline{Y_n^m(\theta, \phi)} \tag{3.6}$$

The surface dipole source representing the BEM double layer formulation requires an operator that is the directional derivative of this basis with respect to the continuous surface element's Cartesian normal vector $n = (n_x, n_y, n_z)$.

To align with the spherical basis, the Cartesian normal is projected into the local spherical components (n_r, n_θ, n_ϕ) through the following standard coordinate transformation:

$$n_r = n_x \sin\theta\cos\phi + n_y \sin\theta\sin\phi + n_z \cos\theta \tag{3.7}$$

$$n_\theta = n_x \cos\theta\cos\phi + n_y \cos\theta\sin\phi - n_z \sin\theta \tag{3.8}$$

$$n_\phi = -n_x \sin\phi + n_y \cos\phi \tag{3.9}$$

The required dipole basis can be constructed as the vector dot product of the spherical spatial gradient and the projected normal vector:

$$\text{dipole\_basis} = \frac{\partial f_n^m}{\partial r} n_r + \frac{1}{r}\frac{\partial f_n^m}{\partial \theta} n_\theta + \frac{1}{r\sin\theta}\frac{\partial f_n^m}{\partial \phi} n_\phi \tag{3.10}$$

The radial gradient component is derived using the derivative recurrence relation for spherical Bessel functions:

$$j_0'(kr) = -k \cdot j_1(kr) \tag{3.11}$$

$$j_n'(kr) = k\left(\frac{n \cdot j_n(kr)}{kr} - j_{n+1}(kr)\right) \text{ for } n > 0 \tag{3.12}$$

To get the angular gradient components, we need to evaluate the derivatives of the spherical harmonics. The polar derivative $\frac{\partial Y_n^m}{\partial \theta}$ is computed using specialized quantum mechanics ladder operators that link adjacent degree coefficients. The inclusion of the

Condon-Shortley phase factor, $(-1)^m$, in the definition of the spherical harmonics is what allows these raising and lowering operators to be applied, vastly improves speed over direct transcendental derivation. The azimuthal derivative $\frac{\partial Y_n^m}{\partial \phi}$ trivially evaluates to imY_n^m.

The FDM Fallback

The analytical P2M method described in 3.4.1 while mathematically exact, can encounter numerical instability when the source point is very close to the expansion center or on its vertical polar axis, as it contains divisions by r and $r \sin\theta$. We have implemented safeguards to detect these edge cases and switch to a finite difference approximation of the normal derivative in these scenarios. The FDM implementation uses two infinitesimally offset target points, $p_+ = \text{center} + \epsilon n$ and $p_- = \text{center} - \epsilon n$, and the analytical basis is evaluated at both these points to compute the derivative numerically via the central difference $(f_+ = f_-)/(2\epsilon)$.

3.4.2. Gaunt Addition Theorem for Acoustic Waves

The M2L operators consolidate coefficients from smaller child nodes into large parent nodes, while the M2L operators convert the outgoing multipole signatures into incoming localized incident fields across the separated nodes of the interaction lists. The M2L and the M2M translations are fundamentally reliant on the exact formulations based on Gaunt addition theorems. The theorem defines the mechanism by which a target multipole coefficient is generated by multiplying the entire spectrum of source coefficients by a specific transformation matrix based on their spatial separation vector d.

Let the displacement vector between the translation centers be defined in local spherical coordinates as (r, θ, ϕ). Following the analytical derivations laid out by Gumerov and Duraiswami [GD09], the specific scalar entry for the transformation matrix (T_{entry}), which links the (n', m') source mode to the (n, m) target mode, requires an internal summation over a third intermediate index p. The bounds of p are strictly dictated by the Wigner triangle inequality $|n - n'| \leq p \leq n + n'$ [GD04].

The matrix entry T_{entry} is accumulated as:

$$T_{entry} = \sum_p 4\pi i^{n'-n+p} \cdot (-1)^{m'} \cdot G(n, m; n', -m'; p) \cdot z_p(kr) \cdot Y_p^{m-m'}(\theta, \phi) \tag{3.13}$$

Each component in this equation has a specific mathematical and physical significance:

- Integration prefactor 4π scales the translation to the appropriate solid angle.

- The complex exponential term $i^{n'-n+p}$ dictates the interference pattern of the translated wave. To avoid having to compute expensive complex exponentials, we leverage the fact that i^k cycles through the values $\{1, i, -1, -i\}$ every four increments of k, allowing us to compute this term with a simple modulo operation and a small lookup table.
- The parity selection rules embedded within the Gaunt coefficient G automatically skip all iterations where $(n + n' + p)$ is an odd integer. This effectively halves the number of required summation loops.
- The term $z_p(kr)$ represents the appropriate spherical radial function, dependent entirely on the operator type.

3.4.3. Formulating the M2M, M2L, and L2L Translation Operators

By simply varying the choice of the spherical radial function $z_p(kr)$, the generic addition theorem described above in 3.4.2 can be used to construct the specific operators required for the FMM passes as shown in Table 3.1.

Operator Type	**Application**	**Target Function** $z_p(kr)$	**Computational Characteristics**
M2M Matrix	Upward Pass (Child to Parent)	Regular Bessel $j_p(kr)$	Translates within a continuous, finite domain. Exact shift of geometric center.
M2L Matrix	Horizontal Pass (Across lists)	Singular Hankel $h_p^{(1)}(kr)$	Translates across the void between well-separated nodes, modeling the required outward acoustic radiation.
L2L Matrix	Downward Pass (Parent to Child)	Regular Bessel $j_p(kr)$	Translates the incident local field deeper into the spatial subdivision.

Table 3.1.: Translation Operators and Computational Characteristics

Since we enforce a fixed uniform expansion order P across the entire tree, the dimensionality of all these translation matrices is locked to $(P+1)^2 \times (P+1)^2$. All these matrices are precomputed once, cached in memory, and applied sequentially to the dynamically changing coefficient vectors during each GMRES iteration.

3.5. Implementing the Upward, Horizontal, and Downward Passes

With our octree generated, the near-field and far-field interaction lists populates, and the dense translation operators fully precomputed, we can now begin the hierarchical traversal of the octree to compute the far-field interactions. This traversal happens in every iteration of iterative solver used to solve the system. There are three distinct, sequential passes in the traversal: the upward pass, the horizontal pass, and the downward pass.

3.5.1. Upward Pass: Aggregating Far-Field Signatures

The traversal begins with the upward pass. It evaluates the initial acoustic influence of the boundary sources contained in the active solution vector(in our case, the ϕ estimated at the previous iteration) in all the leaf nodes. For each leaf node, the P2M operator is multiplied by the specific potential and flux (in our case the normal velocity from our Right-hand side) values assigned to the local boundary elements. This operation yields the multipole expansion coefficients for that leaf node.

The algorithm then traverses up the octree, level by level, from the leaf nodes to the root node. For each parent node, its generalized multipole expansion coefficients are computed by translating the multipole expansions of its child nodes to its own geometric center. Because the M2M operator applies the addition theorem linearly, the parent's total multipole expansion is evaluated as the algebraic sum of the shifted multipole expansions of its children.

$$\mathbf{M}_{parent} = \sum_{c=1}^{\text{child\_count}} \mathbf{T}_{M2M}^{(c)} \cdot \mathbf{M}_{child}^{(c)} \tag{3.14}$$

3.5.2. Horizontal Pass: Translating Multipole Expansions to Local Expansions

The horizontal pass serves as the critical bridge of the FMM algorithm, converting the outgoing scattered radiation represented by the multipole expansions into incoming incident fields represented by the local expansions. For all valid nodes with depth $d \geq 2$, the horizontal pass iterates through the far-field interaction list of the node. For each target-source pairing within the list, the source node's multipole expansion is multiplied by the specific precomputed M2L translation operator associated with the relative displacement vector separating the source and target nodes.

The resulting localized coefficients, which represent the acoustic contribution arriving from distant source nodes are algebraically accumulated into the target node's local coefficient array:

$$\mathbf{L}_{target} \mathrel{+}= \mathbf{T}_{M2L} \cdot \mathbf{M}_{source} \tag{3.15}$$

Due to us enforcing a uniform truncation order P, the dimensions of all the vectors and matrices invovled line up, enabling the use of vectorized matrix-vector multiplication for a significant boost in computational efficiency.

3.5.3. Downward Pass: Distributing Local Expansions to Leaf Nodes

The downward pass cascades the accumulated incident local expansion from the higher levels of the octree down to the leaf nodes, where the final evaluation of the far-field contribution to the acoustic field at the boundary elements takes place. Starting from the nodes on level $d = 2$, and traversing downward, the parent node's aggregated local coefficients are geometrically shifted to the centers of its child nodes using the precomputed L2L translation operator. These shifted coefficients represent the macroscopic background acoustic field, are added to the existing local coefficient the child node accumulated during its horizontal pass interactions. This recursive cascade ensures that when the traversal reaches the leaf nodes, they have a complete representation of the far-field influence from all distant sources.

The final step of the downward pass occurs at the leaf nodes, where the L2P operator transforms the leaf node's local coefficients into the physical acoustic potential and flux (normal velocity) at the specific Cartesian coordinates of the target boundary elements.

Operating in local spherical coordinates relative to the leaf center, the acoustic potential at a target point can be obtained by a straightforward double summation like so:

$$\text{potential} = \sum_{n=0}^{P} \sum_{m=-n}^{n} L_{coeffs}\left[n^2 + n + m\right] \cdot j_n(kr) \cdot Y_n^m(\theta, \phi) \tag{3.16}$$

The normal derivative calculation follows precisely the inverse of the P2M logic. The radial derivative uses the exact stable recurrence $j_n' = k\frac{nj_{n-1} - j_{n+1}}{2n+1}$, and the angular gradients use the same quantum mechanical ladder operators. The final physical normal derivative is generated by the dot product of this resolved spherical gradient vector with the target element's distinct surface normal vector.

Similar to our P2M implementation, if the physical target point is too close to the leaf node center ($r < 10^{-8}$), we fall back to a FDM approach to avoid the singularity. The final assembled physical basis vector output merges both the evaluated potential and the normal derivative, accurately scaled by the prefactor $-ik$ associated with the 3D Helmholtz equation.

3.6. Building the Near-Field Matrix

While the hierarchical translation passes of the FMM efficiently approximate the far-field interactions, the near-field interactions must be computed with high precision to prevent the localized degradation of the solution. The near-field domain for any target node consist of the boundary elements in the node box itself, but also all the boundary elements in its geometrical neighbor boxes, and any coarser leaf boxes that are geometrically adjacent to the target node's parent node.

The acoustic interactions between the boundary elements in the near-field domain are computed directly with the Particle-to-Particle (P2P) mathematical integration. This integration logic is exactly the same as the one used to generate the dense system matrix for the BEM.

4. Software Architecture and HPC Optimizations

4.1. Bitwise Operations for Morton Encoding

As detailed in Section 3.2.2, the Morton Z-order curve is used to map the 3D coordinates of the mesh elements for a more efficient Octree construction. To optimize this process without relying on expensive looping, we use a series of bitwise operations that interleave the bits of the binary representations of the x, y, and z coordinates.

The basic algorithm that we use to compute the Morton key for a given 3D coordinate is as follows:

Step 1. Grid Quantization: We define a discrete grid across the global bounding box. With a 64-bit integer, we can divide each spatial dimension into 2^{20} $(1,048,576)$ bins, which allows us to represent up to 2^{60} unique spatial locations, more than enough for our problem size.

Step 2. Coordinate Normalization: We normalize the x, y, and z coordinates of each mesh element to fit within $[0,1]$ relative to the global bounding box, and then multiply by the number of bins to get the quantized integer coordinates.

Step 3. Bit Truncation: We ensure that the quantized coordinates fit within 21 bits, ensuring that they can be interleaved into a single 64-bit integer without overflow. This is achieved by applying a bitwise `AND` operation with a mask that retains only the lower 21 bits of each coordinate.

Step 4. Bit Expansion: We use a sequence of bitwise shifts and bitwise operations with bit masks to add two zero bits between each bit of the quantized coordinates. This process is repeated for each coordinate.

Step 5. Bit Interleaving: Finally, we combine the expanded bits of each coordinate by shifting the expanded y bits by one, and the expanded z bits by two, and then performing a bitwise OR operation to interleave the bits together, resulting in the final Morton key for that mesh element.

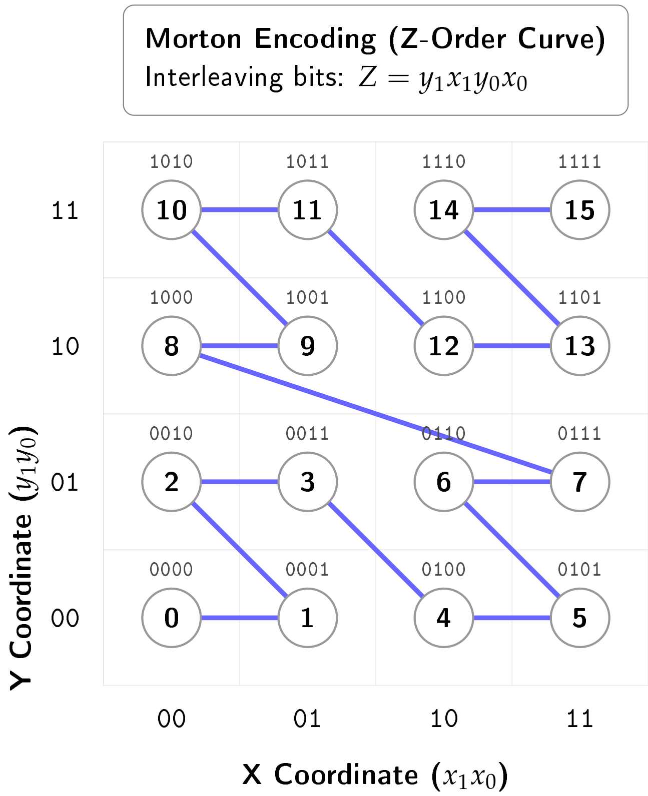


Figure 4.1.: A 2D representation of the Morton Z-order curve. The binary coordinates on the axes represent the quantized and normalized coordinates of the mesh elements. The interleaved binary values above each point represent their corresponding Morton keys.

The basic algorithm of this process is shown for the 2D case in [And05]. The exact implementation for the 3D case is shown in the Appendix A.1

4.2. Caching the Matrices

The FMM requires us to compute a lot of translations matrices, such as the P2M, M2M, M2L, L2L, and L2P matrices. The computation of all these matrices, and especially the M2L matrices, is highly compute-bound. This stems from the fact that calculating the M2L operators requires evaluating the spherical harmonics, associated Legendre polynomials, Wigner 3-j symbols, and the Gaunt coefficients, which make heavy use of recursive branching and high-latency floating point division. This causes both, low arithmetic intensity and severe pipeline stalls, representing a significant bottleneck in the FMM algorithm.

We mitigate this by exploiting the strict geometric symmetry of our uniform Octree. If we consider the geometry of the Octree, we can see that at a particular level, the boundaries of the interaction domain form a localized $7 \times 7 \times 7$ grid of uniform boxes around the target node, as shown in Figure 4.2. Discounting the center box, which is the target node itself, and the 26 boxes that are direct neighbors of the target node, which are handled by direct computation, we are left with $7^3 - 1 - 26 = 316$ possible M2L translation operators that we need to compute per tree level.

Since the number of unique M2L operators is limited to 316 per tree level, this makes it practically feasible for us to precompute and store these matrices. We further optimize this by generating a lookup table that maps each M2L matrix to a transfer ID. To index this lookup table, we rely on the fact that with our $7 \times 7 \times 7$ interaction grid, the relative coordinate offsets in $3D$ space is strictly bounded by the integer range $[-3, 3]$ in each dimension. By shifting this coordinate range by $+3$, we can build our transfer ID Like so:

$$\textbf{Transfer ID} = (\delta x + 3) + 7\left((\delta y + 3) \times 7 + (\delta z + 3)\right)$$

While we allocate memory for all 342 possible M2L matrices, we only compute and store the 316 matrices that correspond to the interaction list. This allows us to seamlessly use the transfer ID as an index into the lookup table, without needing costly floating point comparisons to check if a particular M2L interaction is valid or not. This optimization allows us to skip the extremely expensive computation of the M2L matrices, turning the requirement of performing complex mathematical operations into a trivial $\mathcal{O}(1)$ lookup.

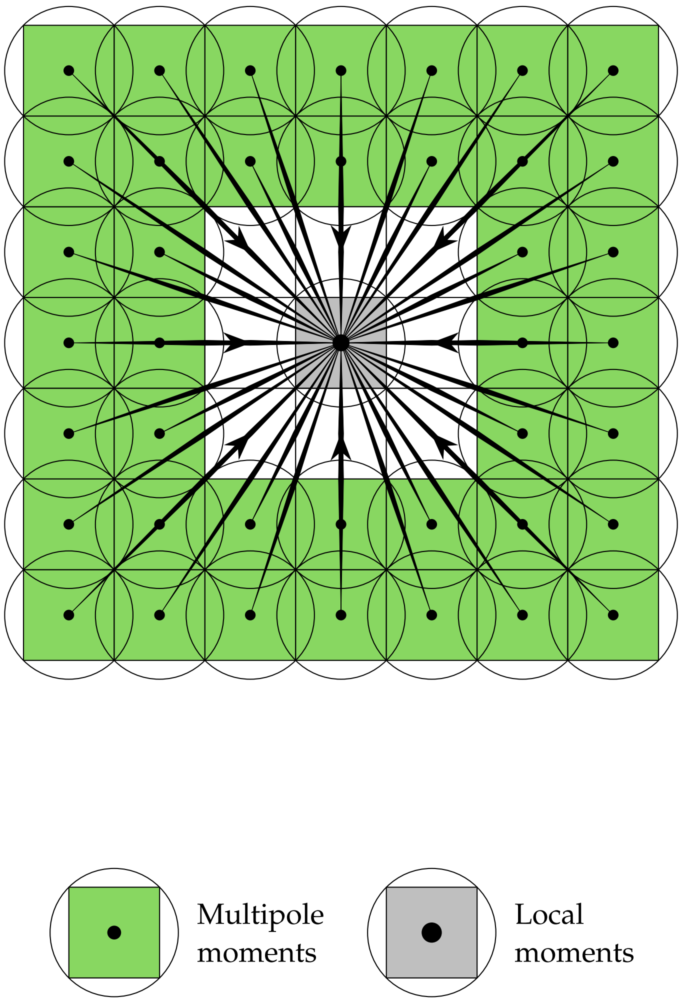


Figure 4.2.: A 2D representation of the M2L interaction list. In 3D, there are $7^3 - 1 = 342$ possible relative positions for the M2L interactions. [Koh+21]

4.3. OpenMP Speed Up

Modern Central Processing Unit (CPU)s are equipped with multiple cores and can execute multiple threads in parallel. On top of that, they support Single Instruction Multiple Data (SIMD) operations, which allows them to perform the same operation on multiple data points simultaneously. The OpenMP Application Programming Interface (API) provides a simple and effective way to parallelize code to take advantage of these features. We leverage OpenMP by setting `#pragma` directives in the region of our code where we want parallelism. These directives instruct the compiler to generate code that can be executed in parallel across multiple threads. OpenMP allows us to control parallelism at both the coarse-grained thread-level and the fine-grained CPU instruction level. The particular strategies we have used in the code are as follows:

4.3.1. Loop Collapsing

Building the global M2L cache as detailed in the previous section helps us bypass particularly troublesome bottleneck. However, there is still more performance to be wrung out of the M2L phase. Computing the global M2L cache requires iterating over the $7 \times 7 \times 7$ grid of possible interactions for each level. This uses three nested loops, which when naïvely parallelized with the standard `#pragma omp parallel for` directive, would only result in a maximum parallel iteration space of 7. Modern CPUs with many more threads available would end up being underutilized. To solve this, and maximize the workload distribution across all cores, we use the `#pragma omp parallel for collapse(3)` directive.

The `collapse(3)` clause tells the compiler to collapse the three hierarchically nested loops into a single loop with a larger iteration space of $7 \times 7 \times 7 = 343$. Each thread then gets assigned a chunk of this larger iteration space, allowing us to fully utilize the available threads and achieve better load balancing.

4.3.2. Dynamic Scheduling

While a highly structured, uniform iteration workload would fit perfectly with OpenMP's default static scheduling, the FMM Octree's workload is inherently irregular. The number of mesh elements in each node can vary widely, especially higher up in the tree where some nodes have stopped subdividing due to a lack of elements. Also, the size of each node's interaction list varies, depending on its spatial location. This leads to a highly imbalanced workload across the nodes, which can cause significant load imbalance when naïvely calling the `#pragma omp parallel for` directive on the Octree traversal loops. The default static scheduling, when called over the `total_nodes`, would divide the total number of nodes by the number of worker threads, would result in considerable load imbalance.

Using the `#pragma omp parallel for schedule(dynamic)` directive is the perfect counter to this. Dynamic scheduling instructs the OpenMP runtime to initialize a centralized work queue, and as worker threads become idle, they pull the next available chunk of iterations from the queue. This dynamic load balancing ensures that no thread sits idle while the work queue is not empty [Klo20].

4.3.3. SIMD Vectorization

Leveraging thread-level parallelism with OpenMP is a great way to speed up the FMM algorithm, but we can take this one step further with instruction-level parallelism. Modern CPUs are equipped with SIMD vector units that enable them to perform the same operation on multiple data points simultaneously [XWZ23]. Instruction sets like the AVX-256 or AVX-512 (Advanced Vector Extensions) use large hardware registers, in the case of the AVX-512, 512 bits wide, which can hold four `std::complex<double>`(64 bits each for the real and imaginary parts) values at once. OpenMP's `#pragma omp simd` allows us to instruct the compiler to generate these specific hardware-level instructions needed to take advantage of these vector units. The general rule for where we can apply this directive is to look for innermost loops that perform simple, predictable arithmetic (especially reductions like `sum += ...`) without branching (`if/else`), function calls, or memory allocations. We use this directive in the innermost loops of the `compute_upward_pass`, `compute_horizontal_pass`, and `compute_downward_pass` functions, which perform the core arithmetic of the FMM algorithm. All these inner loops are of the type `sum += A[i] * B[i]`. These get compiled to the hardware-level Fused Multiply-Add (FMA) instruction, which performs multiplication and addition in a single step, reducing latency and improving performance [Fog22].

4.4. Solving the system - vmult + GMRES + The Real-Equivalent system

The BEM discretization leads to a dense, complex-valued linear system that is complex-symmetric but not Hermitian, and therefore cannot be solved by methods such as the Conjugate Gradient [SB]. While a direct solver can be used, it scales as $\mathcal{O}(N^3)$, which quickly becomes infeasible for large systems. Iterative solvers offer a great alternative, reducing computational cost to $\mathcal{O}(N_{iter}N^2)$, especially when combined with the FMM's shift to matrix-free operator evaluations that does not require storing all N^2 matrix elements [GD09].

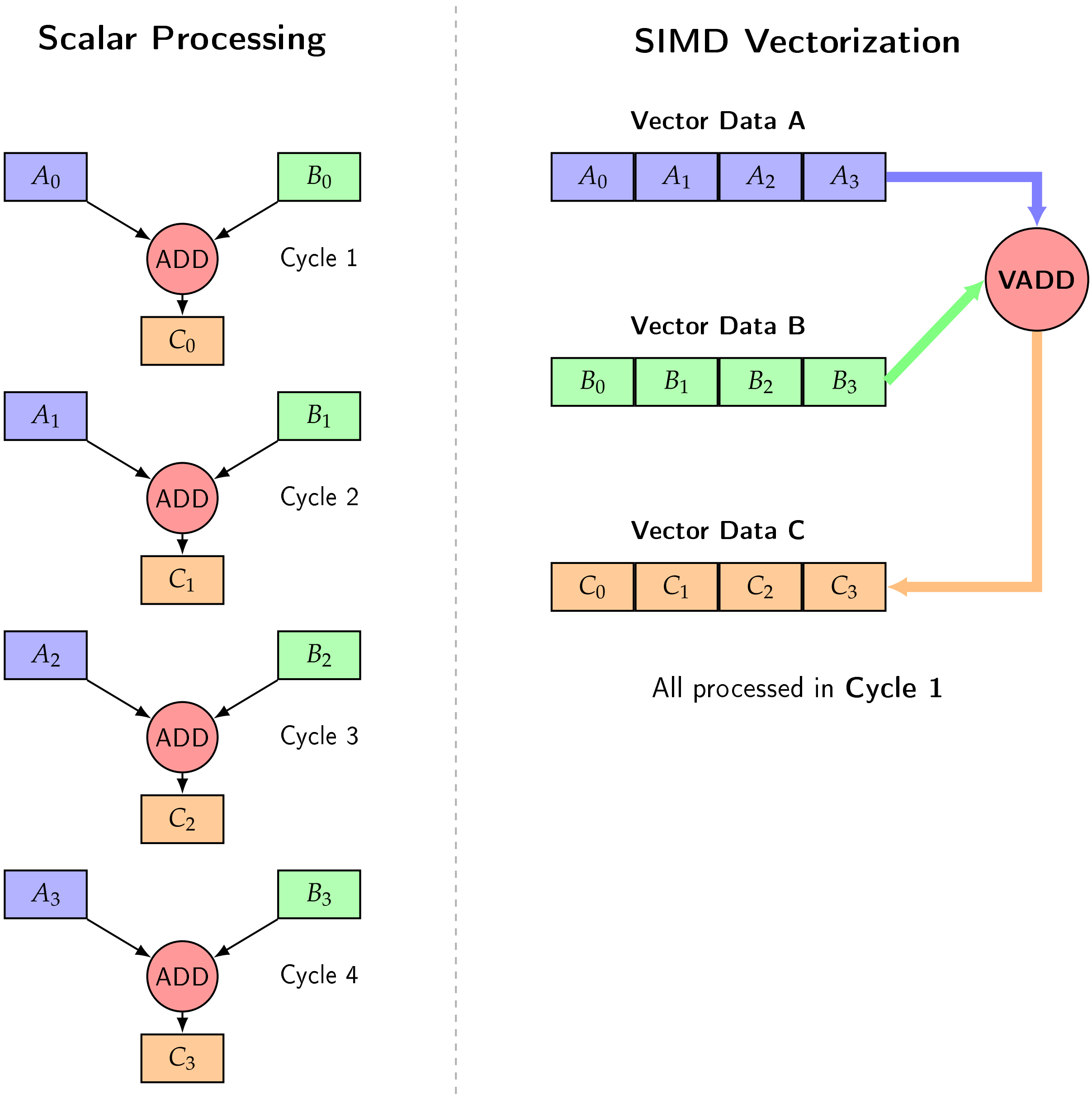


Figure 4.3.: A simplified representation of SIMD vectorization. The same operation is performed on multiple data points simultaneously using wide hardware registers and wide SIMD functional units.

We chose to use the `deal.II` library [Arn+] to implement our BEM and FMM algorithms, as it provides robust and optimized linear algebra and mesh management infrastructure. The `deal.II` library provides a built-in implementation of the GMRES iterative solver. However, it is only intended for real-valued systems [BC]. This limitation is fundamentally a part of the `deal.II` implementation of its `SolverGMRES` [DII:GMRES].

We get around this limitation by formulating our complex-valued system as its equivalent real-valued system [DH01]. The breakdown of our approach is as follows:

1. **Vector Splitting:** We split our complex vectors like `system_rhs` and `phi`, which are of size N, into real `Vector<double>`s of size $2N$, with the first N elements being their real components, and the next N elements being their imaginary parts.

2. **Matrix-Free Wrapper:** We implement a struct `RealFMMOperator` that inherits from `dealii::Subscriptor` and implements a real-valued `vmult` function that the solver expects. It is important to inherit from `dealii::Subscriptor` to ensure that the internal pointers to the FMM data are properly reference-counted and do not lead to memory leaks or dangling pointers.

3. `vmult` **Translations:** Inside our custom `vmult` method, it takes the $2N$-sized real vector and reconstructs the original size N complex vector. It then calls the original complex-valued `vmult` method of our FMM implementation, which performs the matrix-free matrix-vector multiplication. Finally, it takes the resulting complex vector and splits it back into size $2N$ real vector to be returned to the iterative solver.

4. **Post-Processing:** After the GMRES solver converges, we take the resulting size $2N$ real solution vector and reconstruct the complex N-sized `phi` vector, which contains the values of the potential at the collocation points.

Our implementation of the `RealFMMOperator` struct is shown in Appendix A.2.

While iterative Krylov space solvers like GMRES are quite robust in their convergence, convergence depends on the matrix's spectral properties and condition number [SS86]. The real-equivalent formulations can, in theory, produce eigenvalue distributions that are unfavorable for the convergence of GMRES. To ensure fast convergence, we employ a preconditioner to improve the system's spectral properties. The Jacobi-block-diagonal preconditioner [DH01] we implement extracts the complex-scalar diagonal from the near-field sparse matrix we assembled for the FMM and stores its inverse. `Deal.II`'s GMRES, which by default uses left preconditioning, then applies this to the residuals which it tries to minimize. The exact implementation of this preconditioner is shown in Appendix A.3.

5. Performance Analysis and Validation

We have thus far discussed the step-by-step implementation of a FMM-based solver for the BEM formulation of the acoustic scattering problem. In this chapter, we present the results of validating our implementation against analytical solutions, show the effect of using the Burton-Miller formulation, and analyze the performance of our implementation in terms of computational time compared with a traditional BEM solver.

5.1. Validation vs. Analytical Solution

When implementing a new solver, it is critical to validate it against known solutions to ensure it is correctly implemented and produces accurate results. The "acoustic scattering from sound-hard boundaries" problem that we have implemented our solver for has a known analytical solution for a single spherical scatterer in a medium excited by a plane wave. The plane wave is given by:

$$p_{\mathrm{b}} = p_0 e^{-i(\mathbf{k}\cdot\mathbf{x})} = p_0 e^{-ik_s\left(\frac{\mathbf{x}\cdot\mathbf{e}_k}{||\mathbf{e}_k||}\right)}$$

where p_0 is the amplitude of the plane wave, $\mathbf{k}$ is the wave vector with amplitude k_s, $\mathbf{e}_k$ is the unit vector in the direction of wave propagation, and $\mathbf{x}$ is the position vector.

The analytical solution for the scattered pressure field is given as a series expansion in terms of spherical harmonics and spherical Bessel functions, as follows:

$$p_s = p_0 \left\{ \sum_{n=0}^{\infty} \mathrm{i}^n (2n+1) \frac{j_n'(kR)}{h_n'(kR)} P_n(\cos\theta) h_n(kr) \right\}$$

where p_0 is the amplitude of the incident plane wave, j_n and h_n are the spherical Bessel and Hankel functions of the first kind, respectively, R is the radius of the spherical scatterer, P_n is the Legendre polynomial of degree n, θ is the angle between the direction of wave propagation and the position vector of the external point $\mathbf{x}$ we are computing the scattered field at, and r is defined as the distance $r = |\mathbf{x} - \mathbf{x}_0|$, with $\mathbf{x}_0$ being the center of the spherical scatterer [Mar16].

While the series is infinite, in practice we can truncate it at a finite number of terms, N, to obtain an approximation of the scattered field. Our implementation uses a slightly modified version of an empirical formula as shown by [Wis80] to determine the number of terms we use:

$$N_{\text{max}} = \left\lceil rka + 4(ka)^{\frac{1}{3}} + 10 \right\rceil$$

where ka is the size parameter, defined as the product of the wavenumber k and the radius of the scatterer a.

We compare the results of our implementation with the analytical solution using a `Python` script that computes the analytical solution at all collocation points on the scatterer's surface, and then compares these analytical values with the numerical solution obtained from our FMM implementation. We run our solver for a single spherical scatterer with the following parameters:

- **Radius** a**:** 0.1 m
- **Number of elements** N**:** 1533 quad elements
- **Frequency** f**:** 500 Hz
- **Speed of sound** c**:** 343 m/s
- **Wave direction** $\mathbf{e}_k$**:** [1, 0, 0], i.e., the wave is propagating in the positive x-direction.

This results in a size parameter of $ka = 9.159\,162 \times 10^{-1}$, which gives us $N_{\text{max}} = 14$ using our particular empirical criterion. We find that the results from our implementation match the analytical solution with a relative L_2 error, calculated as $\frac{\|p_{\text{num}} - p_{\text{ana}}\|_2}{\|p_{\text{ana}}\|_2}$ of $1.062\,840 \times 10^{-2}$ for the absolute pressure values at the collocation points.

Table 5.1.: Error metrics vs. Analytical Solution for scattering from a rigid sphere ($a = 0.1$ m, $f = 500$ Hz, $ka \approx 0.92$).

Domain	L_2 **Relative Error**	**Max Absolute Error (**L_∞**)**	**Mean Magnitude Error**
Surface	1.06×10^{-2} (1.06%)	2.66×10^{-2}	6.30×10^{-3}
Slice ($z = 0$**)**	5.61×10^{-4} (0.05%)	7.24×10^{-3}	2.22×10^{-4}

5.2. Effect of the Burton Miller Formulation

As we have discussed in Section 2.3, the BM formulation is used to mitigate the non-existence of unique solutions at certain characteristic frequencies. To demonstrate

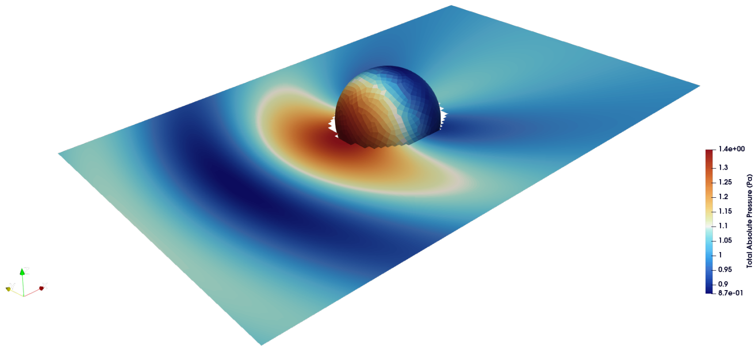


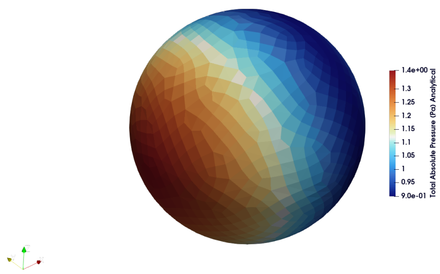
Figure 5.1.: Numerical solution for the absolute pressure field on a slice on the $z = 0$ plane and on the scatterer itself.

the effect of using the BM formulation, we implemented it in our code and toggled it on and off via a command-line flag at runtime. We ran the solver for a sweep of frequencies from $100Hz$ to $3000Hz$ for a single spherical scatterer with the same parameters as in Section 5.1, and compared the results with the analytical solution. We especially include the characteristic frequencies, which for a sound-hard sphere are the Dirichlet Resonances, given by the roots of the spherical Bessel function of the first kind, $j_n(ka) = 0$. Our test script has the ka values corresponding to the resonances present within the range of our frequency sweep hardcoded. These fictitious values lying within $100Hz$ to $3000Hz$ are:

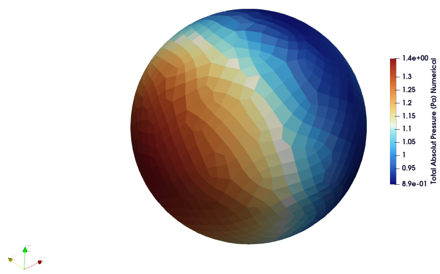
1. $f = 1715Hz$, which corresponds to the first harmonic of a $0.1m$ radius sphere.
2. $f = 2453Hz$, which is the first dipole resonance corresponding to a wavenumber $ka = 4.49341$.

[Kla+19].

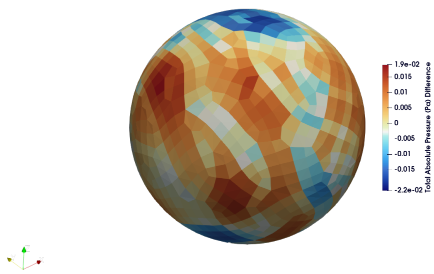
The non BM frequency sweep Figure 5.3 shows the spikes at the characteristic frequencies, while the one with BM Figure 5.4 shows that the error at these frequencies is in line with errors at neighboring frequencies, demonstrating the effectiveness of the BM formulation in eliminating the non-uniqueness issue at the characteristic frequencies. We can also see that the overall error with the BM formulation is slightly higher than without. This is expected, as the BM introduces the HBIE to the system. As the HBIE has a stronger singularity than the BIE, the numerical quadrature errors arising from the singularity of the HBIE kernel are higher than those arising from the



(a) Analytical Solution



(b) Numerical Solution (BEM)



(c) Absolute Error Map

Figure 5.2.: Comparison of the surface pressure fields for acoustic scattering by a rigid sphere ($a = 0.1$ m, $f = 500$ Hz, $ka \approx 0.92$). The analytical partial wave series (a) and the BEM numerical solution (b) show excellent visual agreement. The absolute error map (c) isolates the local discretization error, which peaks at a magnitude of 0.026.

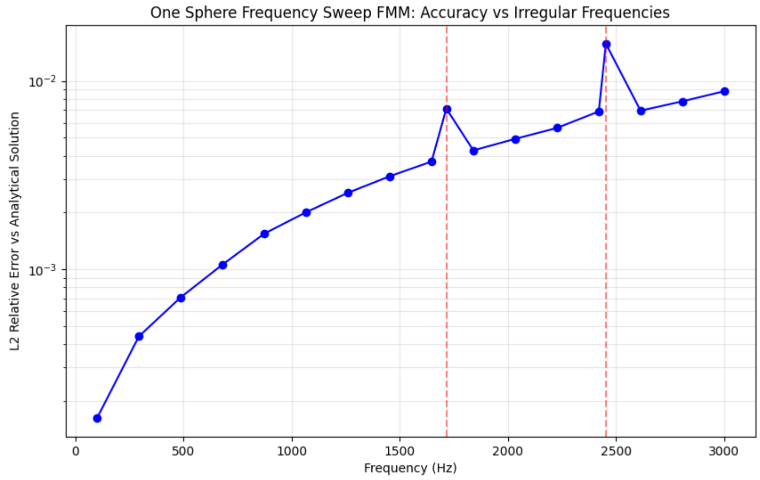


Figure 5.3.: L2 error plot for the FMM without BM formulation.

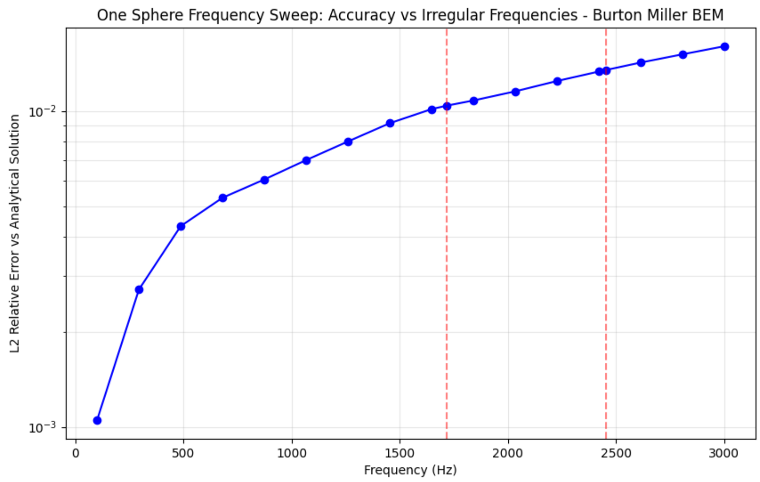


Figure 5.4.: L2 error plot for the FMM with BM formulation.

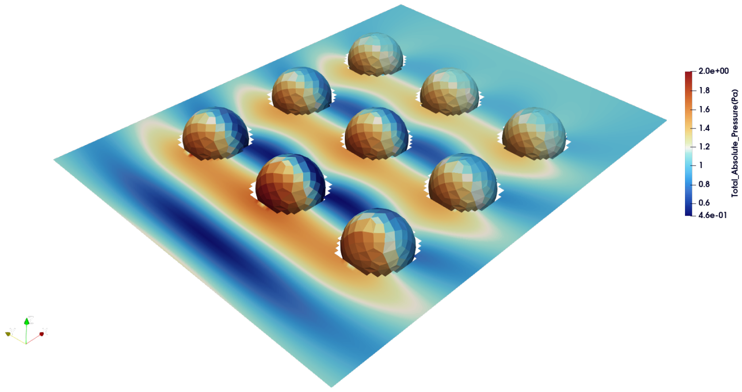


Figure 5.5.: A 3×3 grid of spherical scatterers, in an incident plane wave of frequency $f = 500Hz$. The plot shows the Total Absolute Pressure (Pa) field on the surface of the scatterers, as well as a slice of the field on the $z = 0$ plane.

CBIE kernel, leading to a higher overall error for the BM formulation compared to the non-BM formulation.

5.3. Performance Analysis

The main motivation for implementing the FMM is that it offers much better scaling as compared to the traditional BEM approach. Traditional BEM with direct solvers has a computational complexity of $\mathcal{O}(N^3)$. While the FMM has a computational complexity of $\mathcal{O}(N^{\frac{3}{2}})$ or even $\mathcal{O}(N \log N)$ for a multilevel implementation with iterative solvers. This means that even with the additional overhead of managing the hierarchical structure, the FMM will break even with, and surpass the performance of the BEM. To test the scaling of our FMM implementation, we run both the FMM and a traditional BEM solver on a series of meshes with a grid of identical spherical scatterers, increasing the number of spheres from 1 to 169. We plot the time to set up the system matrix for the traditional BEM, compared with the time to set up the FMM data structures and precompute the translation operators. We compare the time taken by the direct solver for the traditional BEM with the time taken by the GMRES solver for the FMM. Lastly, we plot the total time taken by each method as a whole.

Add the non OpenMP plots

In all these timing plots Figure 5.6, Figure 5.7, and Figure 5.8, we have included guidelines corresponding to the closest asymptotic complexity for each timing curve. Let us consider the traditional BEM first. The time taken to set up the system matrix scales as $\mathcal{O}(N^2)$, as we have to compute the interaction between every pair of elements.

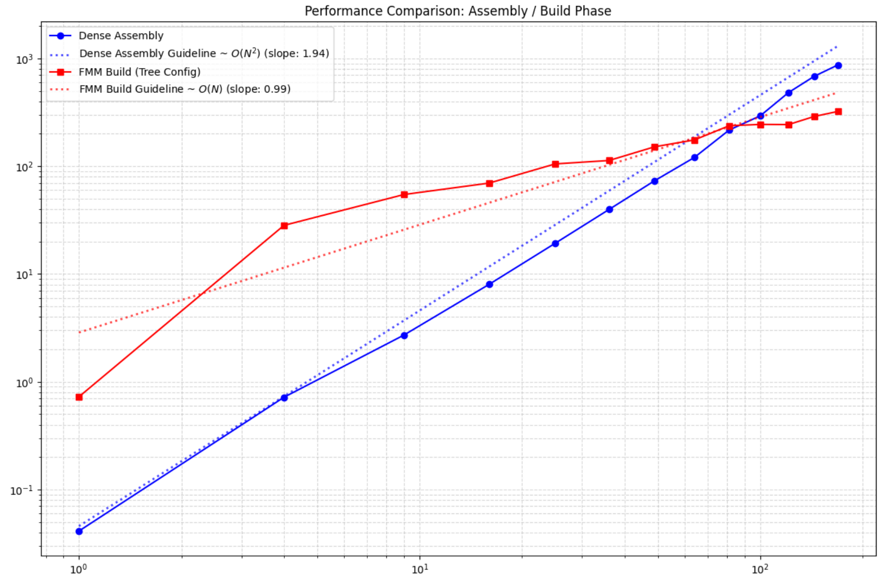


Figure 5.6.: Time taken to set up the system matrix for the traditional BEM, as compared to the time taken to set up the FMM data structures and precompute the translation operators.

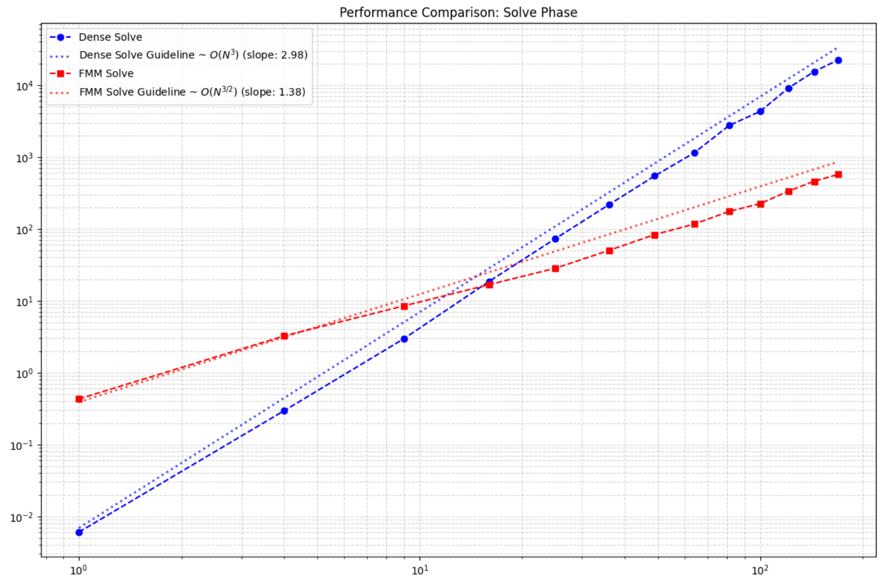


Figure 5.7.: Time taken to solve the system for the traditional BEM, as compared to the time taken to solve the FMM system.

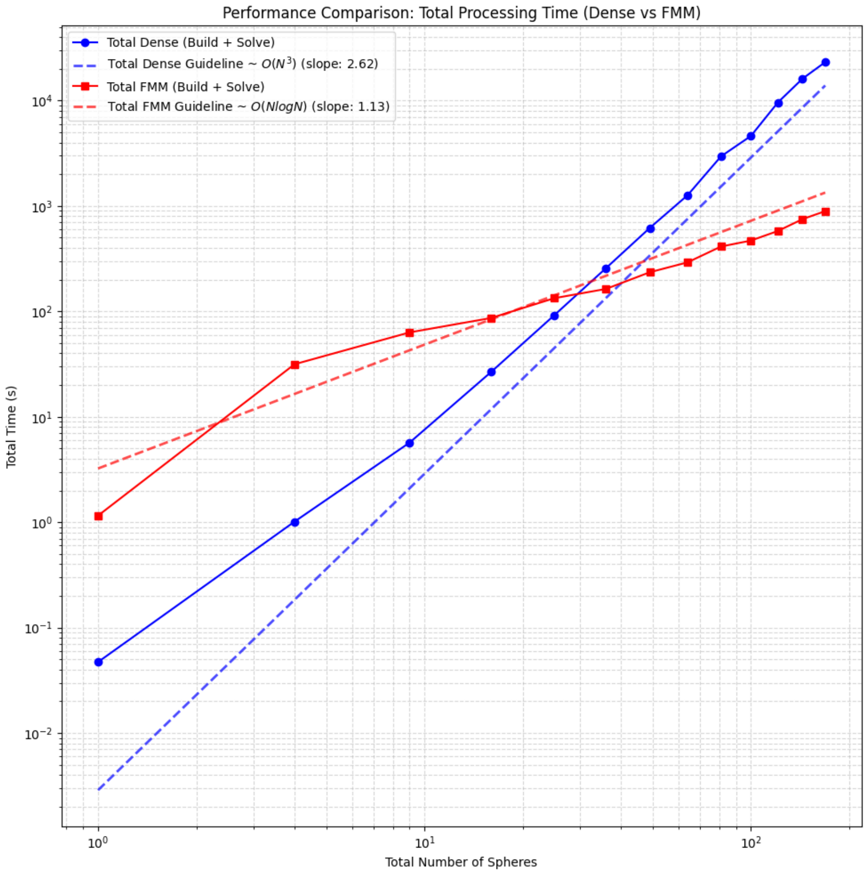


Figure 5.8.: Total time taken by the traditional BEM, as compared to the time taken by the FMM.

The time taken to solve the system scales as $\mathcal{O}(N^3)$, but considering the solving time makes up the bulk of the total time needed for the traditional BEM, the total time also scales as $\mathcal{O}(N^3)$. The FMM build time scales as $\mathcal{O}(N)$. This time is dominated by the time spent computing the M2L translation operators, the number of which scales somewhat linearly with N elements. The solution time taken by the Real-Equivalent system with GMRES scales as $\mathcal{O}(N^{\frac{3}{2}})$. As the amount of overhead needed to set up the FMM data structures and precompute the translation operators goes down as a percentage of the total time taken as N increases, the total time taken by the FMM scales as $\mathcal{O}(N \log N)$.

6. Conclusion and Future Directions

In this work we have developed an implementation of the FMM for the BEM formulation of the Helmholtz equation for sound hard exterior scattering problems. We began with a classical collocation point based implementation of the BEM made with the `deal.II` library, and proceed to implement the BM formulation of the problem, which mitigates error spikes at certain fictitious frequencies. We built an Octree data structure for the FMM that adapatively subdivides the domain into boxes. We added Z-order space filling curves to make tree traversal more efficient. We implemented the translation matrices for the FMM and the tree traversal with the upward, downward, and horizontal passes. We implemented the matrix-vector multiplication for the FMM which is the crux of the algorithm that lets us solve the system without ever having to explicitly assemble the full matrix. We implemented a conversion layer that converts our complex system into the real-equivalent system that allows us to utilize the native `deal.II` implementation of GMRES to iteratively solve the system. Finally, we leveraged OpenMP strategically to parallelize the computation of the translation matrices and the matrix-vector multiplication for shared memory systems.

We validated our implementation against a known analytical solution for plane-wave scattering off of a single sphere. We also demonstrated the effectiveness of the BM formulation in mitigating error spikes at fictitious frequencies. Finally, we demonstrated the measurable improvement in the computational order provided by the FMM over a traditional direct BEM approach.

While our implementation has demonstrated its performance, it l has some limitations that we must acknowledge. The current implementation is not well suited for higher frequencies. We have implemented the original classical multipole expansions using regular spherical harmonics. As the frequency goes up, we need an ever smaller final leaf node size to maintain accuracy, which leads to increasing depth and increasing number of translation matrices that need to be stored. This can be addressed by implementing the plane wave expansion of the multipole expansions, which is more efficient for higher frequencies, but breaks down for lower frequencies. They can be used together in a combined approach as demonstrated by [GD09]. Our BM implementation depends on regularization that arises from the use of a single dof per cell and perfectly flat panels. This is not the case for higher order elements, and so we would need to implement a more general regularization approach to support higher order elements.

We have also built the solver around purely the sound-hard scatterer problem, and this can be extended to support arbitrary impedance boundary conditions. It would also be interesting to use an iterative solver that natively supports complex numbers and compare it to the convergence of the real-equivalent system we use with GMRES. Also, the use of flexible iterative solvers with adjustable preconditioners, where the preconditioning is based on the solution of a coarser FMM itself could be interesting to implement. Finally, we have only implemented a shared memory parallelization strategy with OpenMP, but the FMM lends itself to distributed memory parallelization as well, and this can be implemented with MPI.

Abbreviations

BEM Boundary Element Method

FMM Fast Multipole Method

FEM Finite Element Method

FDM Finite Difference Method

PML Perfectly Matched Layer

BIE Boundary Integral Equation

CPV Cauchy Principal Value

HFP Hadamard Finite Part

BM Burton-Miller

CBIE Conventional Boundary Integral Equation

HBIE Hypersingular Boundary Integral Equation

CHIEF Combined Helmholtz Integral Equation Formulation

M2M Multipole-to-Multipole

M2L Multipole-to-Local

L2L Local-to-Local

L2P Local-to-Particle

P2M Particle-to-Multipole

P2P Particle-to-Particle

GMRES Generalized Minimal Residual Method

CPU Central Processing Unit

API Application Programming Interface

SIMD Single Instruction Multiple Data

FMA Fused Multiply-Add

List of Figures

List of Tables

Bibliography

[And05] S. E. Anderson. *Bit Twiddling Hacks*. Bit Twiddling Hacks. May 5, 2005. URL: `https://graphics.stanford.edu/~seander/bithacks.html#InterleaveBMN` (visited on 03/21/2026).

[Arn+] D. Arndt, W. Bangerth, M. Bergbauer, B. Blais, M. Fehling, R. Gassmöller, T. Heister, L. Heltai, M. Kronbichler, M. Maier, P. Munch, S. Scheuerman, B. Turcksin, S. Uzunbajakau, D. Wells, and M. Wichrowski. "The Deal.II Library, Version 9.7, 2025." In: ().

[BC] W. Bangerth and Y.-Y. Cai. *The Deal.II Library: The Step-58 Tutorial Program*. The deal.II Library. URL: `https://dealii.org/current/doxygen/deal.II/step_58.html` (visited on 03/24/2026).

[BM71] A. J. Burton and G. F. Miller. "The Application of Integral Equation Methods to the Numerical Solution of Some Exterior Boundary-Value Problems." In: *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 323.1553 (June 8, 1971), pp. 201–210. ISSN: 0080-4630, 2053-9169. DOI: `10.1098/rspa.1971.0097`.

[Che01] W. C. Chew, ed. *Fast and Efficient Algorithms in Computational Electromagnetics*. Artech House Antennas and Propagation Library. Boston: Artech House, 2001. 931 pp. ISBN: 978-1-58053-152-8.

[CL07] S. Chandler-Wilde and S. Langdon. "Boundary Element Methods for Acoustics." In: (July 18, 2007).

[CRA90] C. C. Chien, H. Rajiyah, and S. N. Atluri. "An Effective Method for Solving the Hyper-Singular Integral Equations in 3-D Acoustics." In: *The Journal of the Acoustical Society of America* 88.2 (Aug. 1, 1990), pp. 918–937. ISSN: 0001-4966, 1520-8524. DOI: `10.1121/1.399743`.

[CRW93] R. Coifman, V. Rokhlin, and S. Wandzura. "The Fast Multipole Method for the Wave Equation: A Pedestrian Prescription." In: *IEEE Antennas and Propagation Magazine* 35.3 (June 1993), pp. 7–12. ISSN: 1045-9243. DOI: `10.1109/74.250128`.

[DH01] D. Day and M. A. Heroux. "Solving Complex-Valued Linear Systems via Equivalent Real Formulations." In: *SIAM Journal on Scientific Computing* 23.2 (Jan. 2001), pp. 480–498. ISSN: 1064-8275, 1095-7197. DOI: 10.1137/S1064827500372262.

[DII:GMRES] *The Deal.II Library: SolverGMRES$<$ VectorType $>$ Class Template Reference*. URL: https://dealii.org/9.4.1/doxygen/deal.II/classSolverGMRES.html (visited on 03/25/2026).

[Fog22] A. Fog. "VCL C++ Vector Class Library Manual." In: (Aug. 7, 2022).

[GD04] N. A. Gumerov and R. Duraiswami. "Recursions for the Computation of Multipole Translation and Rotation Coefficients for the 3-D Helmholtz Equation." In: *SIAM Journal on Scientific Computing* 25.4 (Jan. 2004), pp. 1344–1381. ISSN: 1064-8275, 1095-7197. DOI: 10.1137/S1064827501399705.

[GD09] N. A. Gumerov and R. Duraiswami. "A Broadband Fast Multipole Accelerated Boundary Element Method for the Three Dimensional Helmholtz Equation." In: *The Journal of the Acoustical Society of America* 125.1 (Jan. 1, 2009), pp. 191–205. ISSN: 0001-4966, 1520-8524. DOI: 10.1121/1.3021297.

[GR87] L. Greengard and V. Rokhlin. "A Fast Algorithm for Particle Simulations." In: *Journal of Computational Physics* 73.2 (Dec. 1987), pp. 325–348. ISSN: 00219991. DOI: 10.1016/0021-9991(87)90140-9.

[Kir98] S. Kirkup. *The Boundary Element Method in Acoustics: A Development in Fortran*. Integral Equation Methods in Engineering 1. Hebden Bridge: Integrated Sound Software, 1998. 136 pp. ISBN: 978-0-9534031-0-3.

[Kla+19] E. Klaseboer, F. D. Charlet, B.-C. Khoo, Q. Sun, and D. Y. Chan. "Eliminating the Fictitious Frequency Problem in BEM Solutions of the External Helmholtz Equation." In: *Engineering Analysis with Boundary Elements* 109 (Dec. 2019), pp. 106–116. ISSN: 09557997. DOI: 10.1016/j.enganabound.2019.06.021.

[Klo20] K. Kloberdanz. *OpenMP - Static or Dynamic Scheduling?* May 10, 2020. URL: https://kkloberdanz.github.io/2020-05-10.html (visited on 03/24/2026).

[Koh+21] B. Kohnke, C. Kutzner, A. Beckmann, G. Lube, I. Kabadshow, H. Dachsel, and H. Grubmüller. "A CUDA Fast Multipole Method with Highly Efficient M2L Far Field Evaluation." In: *The International Journal* *of High Performance Computing Applications* 35.1 (Jan. 2021), pp. 97–117. ISSN: 1094-3420, 1741-2846. DOI: 10.1177/1094342020964857.

[LGB15] C. Langrenne, A. Garcia, and M. Bonnet. "Solving the Hypersingular Boundary Integral Equation for the Burton and Miller Formulation." In: *The Journal of the Acoustical Society of America* 138.5 (Nov. 1, 2015), pp. 3332–3340. ISSN: 0001-4966, 1520-8524. DOI: 10.1121/1.4935134.

[Liu09] Y. Liu. *Fast Multipole Boundary Element Method: THEORY AND APPLICATIONS IN ENGINEERING*. CAMBRIDGE UNIVERSITY PRESS, 2009.

[LN06] Y. Liu and N. Nishimura. "The Fast Multipole Boundary Element Method for Potential Problems: A Tutorial." In: *Engineering Analysis with Boundary Elements* 30.5 (May 2006), pp. 371–381. ISSN: 09557997. DOI: 10.1016/j.enganabound.2005.11.006.

[Mar08] S. Marburg. "Boundary Element Method for Time-Harmonic Acoustic Problems." In: *Computational Acoustics of Noise Propagation in Fluids*. Ed. by S. Marburg and B. Nolte. Berlin, Heidelberg: Springer, 2008.

[Mar16] S. Marburg. "The Burton and Miller Method: Unlocking Another Mystery of Its Coupling Parameter." In: *Journal of Computational Acoustics* 24.01 (Mar. 2016), p. 1550016. ISSN: 0218-396X, 1793-6489. DOI: 10.1142/S0218396X15500162.

[MB15] D. Malhotra and G. Biros. "PVFMM: A Parallel Kernel Independent FMM for Particle and Volume Potentials." In: *Communications in Computational Physics* 18.3 (Sept. 2015), pp. 808–830. ISSN: 1815-2406, 1991-7120. DOI: 10.4208/cicp.020215.150515sw.

[Rok90] V. Rokhlin. "Rapid Solution of Integral Equations of Scattering Theory in Two Dimensions." In: *Journal of Computational Physics* 86.2 (Feb. 1990), pp. 414–439. ISSN: 00219991. DOI: 10.1016/0021-9991(90)90107-C.

[SB] J. J. Shirron and I. Babuska. "A Comparison of Approximate Boundary Conditions and Infinite Methods for Exterior Helmholtz Problems." In: ().

[Sch68] H. A. Schenck. "Improved Integral Formulation for Acoustic Radiation Problems." In: *The Journal of the Acoustical Society of America* 44.1 (July 1, 1968), pp. 41–58. ISSN: 0001-4966, 1520-8524. DOI: 10.1121/1.1911085.

[SK23] Q. Sun and E. Klaseboer. "Non-Singular Burton–Miller Boundary Element Method for Acoustics." In: *Fluids* 8.2 (Feb. 5, 2023), p. 56. ISSN: 2311-5521. DOI: 10.3390/fluids8020056.

[SS86] Y. Saad and M. H. Schultz. "GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems." In: *SIAM Journal on Scientific and Statistical Computing* 7.3 (July 1986), pp. 856–869. ISSN: 0196-5204, 2168-3417. DOI: 10.1137/0907058.

[Ter] T. Terai. "ON CALCULATION OF SOUND FIELDS AROUND THREE DLMENSIONAL OBJECTS BY INTEGRAL. EQUATION MkTt 1()DS." In: ().

[Wis80] W. J. Wiscombe. "Improved Mie Scattering Algorithms." In: *Applied Optics* 19.9 (May 1, 1980), p. 1505. ISSN: 0003-6935, 1539-4522. DOI: 10.1364/AO.19.001505.

[Wu00] T. Wu. *Boundary Element Acoustics: Fundamentals and Computer Codes*. Southampton, UK: WIT Press, 2000.

[XWZ23] C. Xie, H. Wu, and J. Zhou. "Vectorization Programming Based on HR DSP Using SIMD." In: *Electronics* 12.13 (July 3, 2023), p. 2922. ISSN: 2079-9292. DOI: 10.3390/electronics12132922.

A. Appendix: Code

A.1. Morton Encoding

```
// Expands a 21 bit integer into 63 bits by inserting 2 zeros between each bit
inline uint64_t expand_bits_3d(uint32_t v) {
    uint64_t x = v & 0x1FFFFF;
    x = (x | (x << 32)) & 0x1F00000000FFFF;
    x = (x | (x << 16)) & 0x1F0000FF0000FF;
    x = (x | (x << 8)) & 0x100F00F00F00F00F;
    x = (x | (x << 4)) & 0x10c30c30c30c30c3;
    x = (x | (x << 2)) & 0x1249249249249249;
    return x;
}
inline uint64_t morton_code_3d(uint32_t x, uint32_t y, uint32_t z) {
    return expand_bits_3d(x) | (expand_bits_3d(y) << 1) | (expand_bits_3d(z) << 2);
}
```

Listing A.1: Morton Encoding for 3D

A.2. Real-Equivalent System

```
struct RealFMMOperator : public Subscriptor
  {
    const FMMBEMOperator<dim - 1, dim> &complex_op;
    mutable Vector<std::complex<double>> c_src;
    mutable Vector<std::complex<double>> c_dst;

    RealFMMOperator(const FMMBEMOperator<dim - 1, dim> &op)
      : complex_op(op) {}

    void vmult(Vector<double> &dst, const Vector<double> &src) const
    {
      const unsigned int n_dofs = src.size() / 2;

      if (c_src.size() != n_dofs)
      {
        c_src.reinit(n_dofs);
        c_dst.reinit(n_dofs);
      }

      for (unsigned int i = 0; i < n_dofs; ++i)
      {
        c_src[i] = std::complex<double>(src[i], src[i + n_dofs]);
      }

      complex_op.vmult(c_dst, c_src);

      for (unsigned int i = 0; i < n_dofs; ++i)
      {
        dst[i] = c_dst[i].real();
        dst[i + n_dofs] = c_dst[i].imag();
      }
    }
  } real_fmm_operator(fmm_operator);
```

Listing A.2: Conversion to Real-Equivalent System

A.3. Preconditioning for GMRES

```
class RealDiagonalPreconditioner : public Subscriptor
{
private:
    unsigned int N;
    std::vector<std::complex<double>> inv_diag;

public:
    RealDiagonalPreconditioner(const SparseMatrix<std::complex<double>> &mat)
    {
        N = mat.m();
        inv_diag.resize(N);
        for (unsigned int i = 0; i < N; ++i)
        {
            // Extract the exact diagonal element from our near-field matrix
            std::complex<double> diag_val = mat(i, i);

            // Prevent division by zero
            if (std::abs(diag_val) < 1e-14)
            {
                inv_diag[i] = 1.0;
            }
            else
            {
                inv_diag[i] = 1.0 / diag_val;
            }
        }
    }

    void vmult(Vector<double> &dst, const Vector<double> &src) const
    {
        // Apply the inverse diagonal block to the 2N vector
        for (unsigned int i = 0; i < N; ++i)
        {
            std::complex<double> val(src[i], src[i + N]);
            std::complex<double> precond_val = val * inv_diag[i];
```

```
            dst[i] = precond_val.real();
            dst[i + N] = precond_val.imag();
        }
    }
};
```

Listing A.3: Preconditioning for GMRES